

ANÁLISIS EXPLORATORIO DE DATOS: Visualizaciones

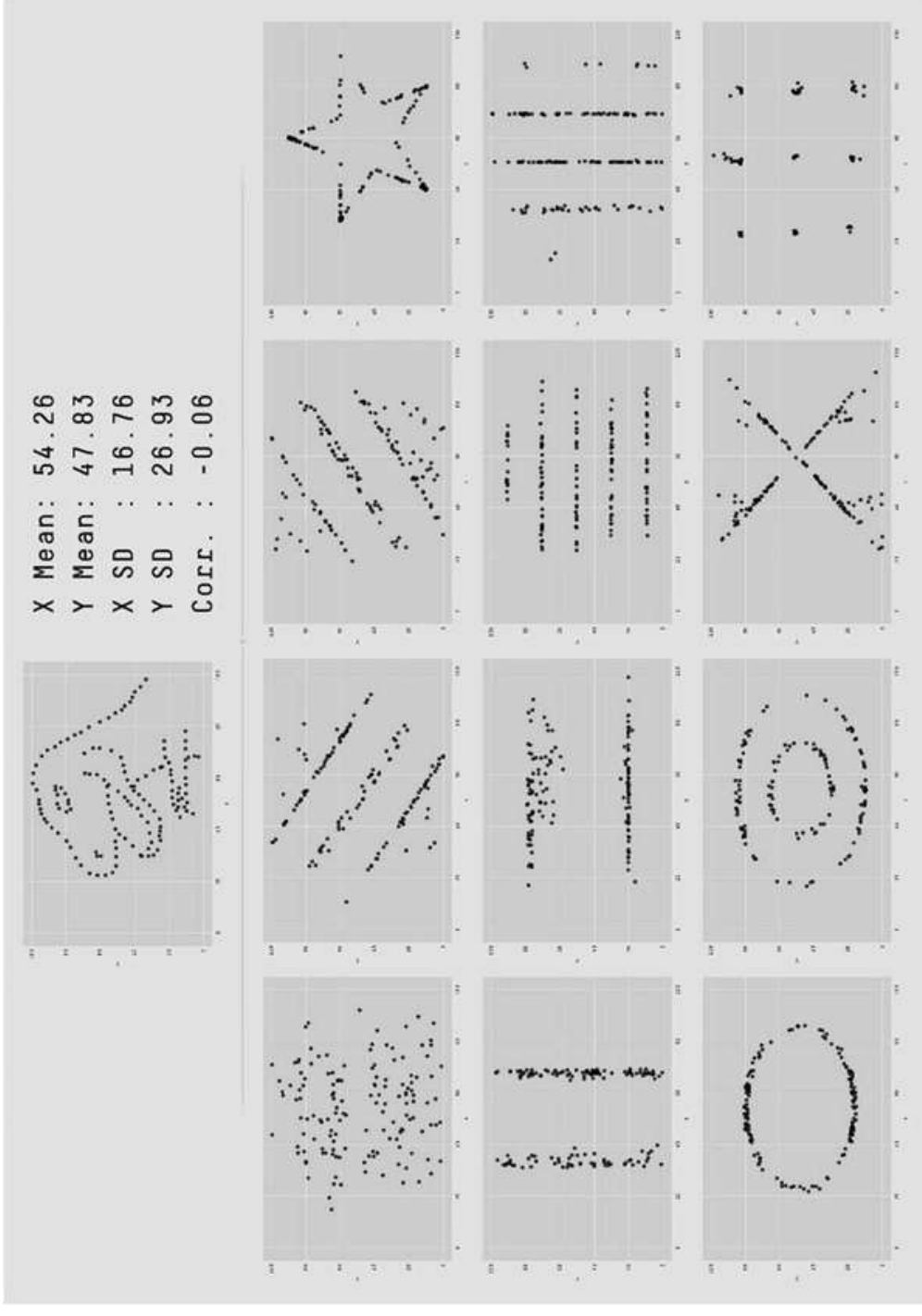
UNIDAD 4 - Parte 1

Fundamentos de Ciencia de Datos 2026



¿Por qué visualizar? El Datasaurus Dozen

El cálculo de **estadísticas** no nos permite contar la historia completa de los datos con los que trabajamos.



¿Por qué visualizar?

La visualización de datos es mucho más que construir gráficos. Es una forma de representar información visualmente con el objetivo de **generar conocimiento**.

Las herramientas de visualización ayudan a poner en evidencia información relevante sobre un dataset, facilitando la detección de **tendencias, patrones, valores atípicos y correlaciones entre variables**.

Además, la visualización cumple un rol central en la **comunicación de resultados**.

Herramientas de visualización en Python

MATPLOTLIB. Es la biblioteca base de visualización en Python. Su módulo más utilizado es **pyplot**.



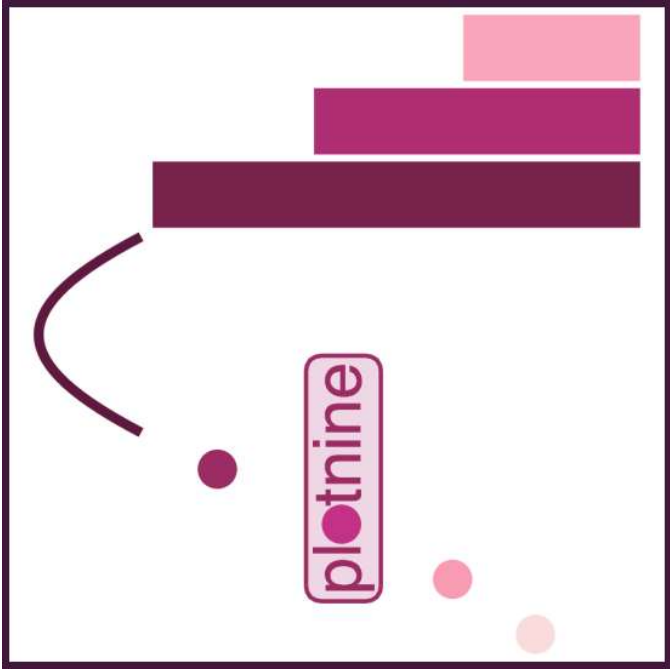
Herramientas de visualización en Python

SEABORN. Es una librería de gráficos de alto nivel construida sobre **matplotlib** que simplifica la creación de diversas visualizaciones de uso común.



seaborn

Herramientas de visualización en Python



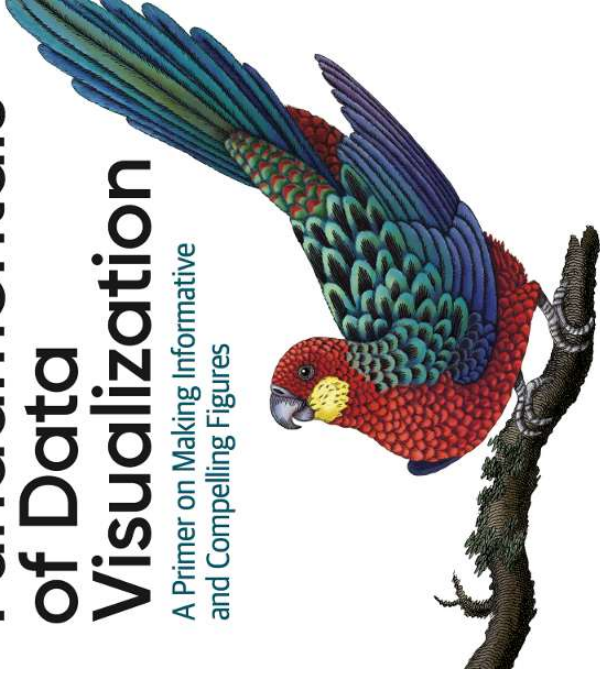
PLOTNINE. Es una implementación de *Grammar of Graphics* (libro escrito por Leland Wilkinson en 1999) en Python, basada en **ggplot2** de R. La *gramática de los gráficos* de Wilkinson provee un sistema para combinar elementos gráficos que den como resultado figuras para mostrar datos de manera visualmente significativa.

Clasificación según Claus Wilke

O'REILLY®

Fundamentals of Data Visualization

A Primer on Making Informative
and Compelling Figures



Claus O. Wilke

Claus Wilke propone una clasificación de las herramientas de visualización según el objetivo que se persigue. Por ejemplo:

- Gráficos para **visualizar cantidades y proporciones**
- Gráficos para **visualizar distribuciones**
- Gráficos para **visualizar relaciones entre dos variables cuantitativas**
- Gráficos para **datos geospaciales**

GRÁFICOS PARA VISUALIZAR DISTRIBUCIONES

GRÁFICOS PARA VISUALIZAR DISTRIBUCIONES



¿De qué hablamos cuando hablamos de DISTRIBUCIÓN?

Cuando hablamos de la **distribución de las observaciones de una variable**, nos referimos a cómo se reparten sus valores: qué valores aparecen, con qué frecuencia y en qué rangos se concentran.

Recuperemos el dataset de la **Encuesta Nacional de Factores de Riesgo (ENFR 2018)** con el que trabajamos en la Unidad anterior.

GRÁFICOS PARA VISUALIZAR DISTRIBUCIONES

Supongamos, en primer lugar, que deseamos construir un gráfico para visualizar la distribución de las observaciones de la ***cantidad de miembros del hogar*** (***cant_componentes***). ¿Qué características tienen las observaciones de esta variable?

```
# ¿Cuántos valores únicos tenemos?  
df_enfr['cant_componentes'].unique()  
  
array([ 2,  3,  1,  4,  6,  5,  7,  8, 10,  9, 11, 14, 12, 13, 15],  
      dtype=int64)
```

Aunque contamos con un gran número de observaciones ($n > 29000$) Vemos que hay **relativamente pocos valores diferentes** de esta variable.

GRÁFICO DE BASTONES

Cuando el número de observaciones es grande pero hay **pocos valores diferentes**, como ocurre muchas veces cuando se trabaja con **variables cuantitativas discretas**, el **gráfico o diagrama de bastones** es la representación gráfica adecuada en este caso.

En el eje de abscisas se representan los **valores observados de la variable** y en el de ordenadas, las **correspondientes frecuencias** (absolutas o relativas). Luego, para cada valor observado se levanta **un segmento o bastón** de altura igual a su frecuencia.

GRÁFICO DE BASTONES

Podemos construir un gráfico de este tipo utilizando la función **countplot()** de la librería **seaborn**, seteando el parámetro **width** en un valor pequeño de manera que lo que se muestren sean **bastones** y no barras.

La variable que se muestra en el eje *y* puede modificarse a través del parámetro **stat**, pudiendo ser: **count** (*default*), **percent** o **proportion**.

GRÁFICO DE BASTONES

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.countplot(x = 'cant_componentes', stat = 'percent', width = 0.2, data
```

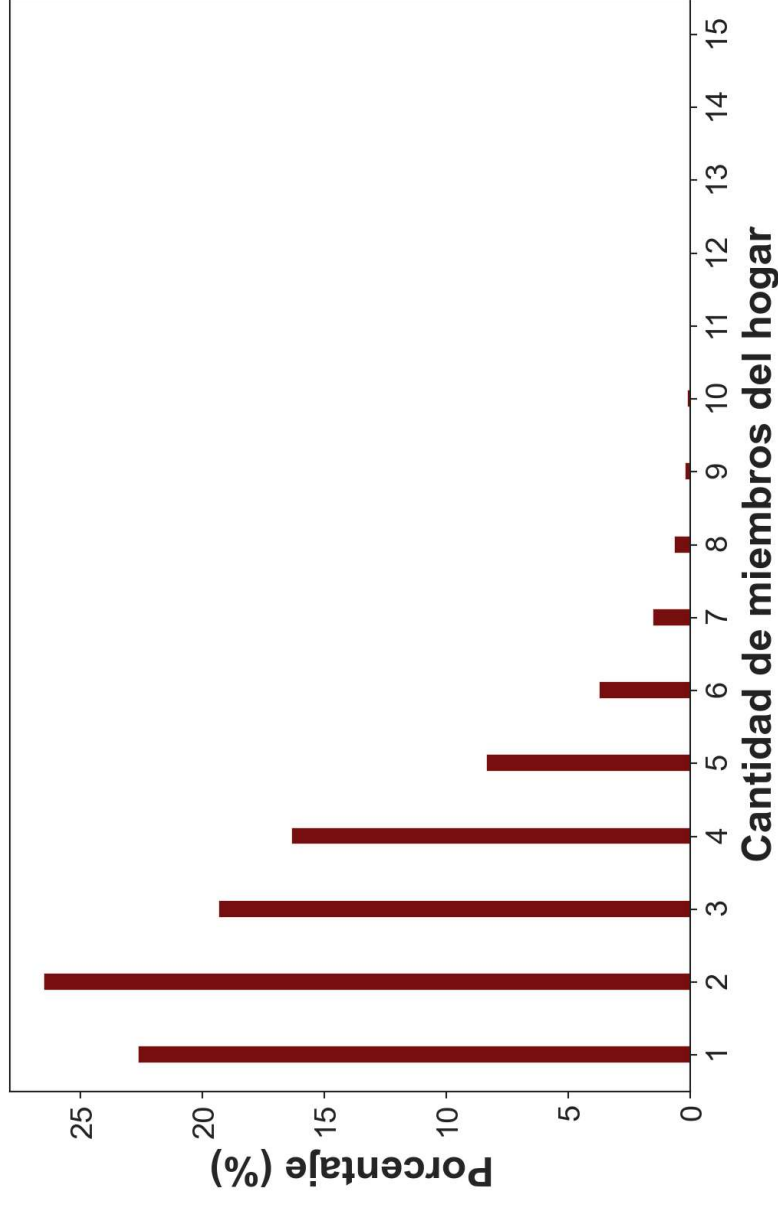


GRÁFICO DE BASTONES

El gráfico anterior es la representación visual de la correspondiente tabla:

```
tabla = (df_enfr['cant_componentes'].value_counts(normalize = True) * 100).sort_index()
print(tabla)
```

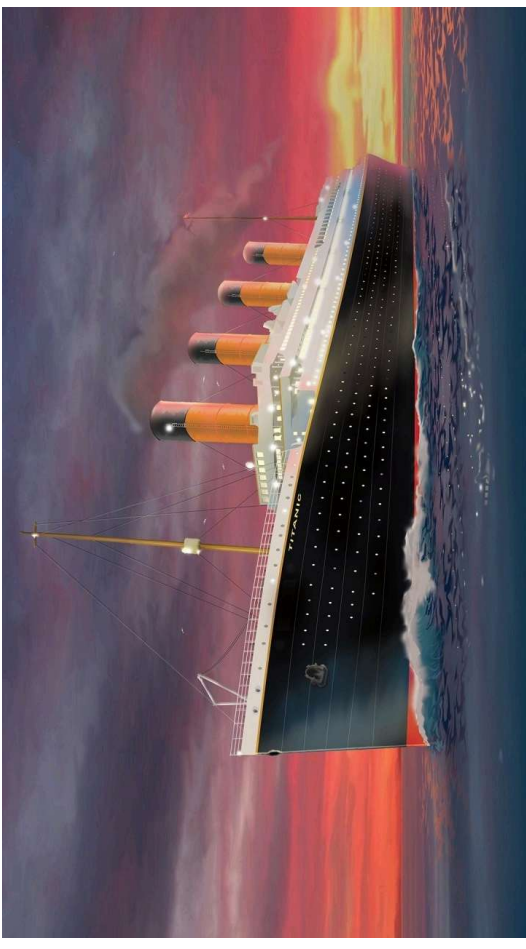
	cant_componentes	porcentaje
0	1	22.69
1	2	26.55
2	3	19.38
3	4	16.38
4	5	8.41
5	6	3.77
6	7	1.57
7	8	0.70
8	9	0.00

DATASET TITANIC

A continuación, vamos a trabajar con el dataset **titanic**. El mismo viene incorporado en **seaborn** y contiene información relacionada con cada pasajero del barco (edad, sexo, clase del pasaje, precio pagado, si el pasajero sobrevivió o no, etc.).

Podemos cargar el dataset en el entorno de trabajo utilizando el siguiente comando:

```
df_titanic = sns.load_dataset('titanic')
```



DATASET TITANIC - Variable age

¿Qué características tienen las observaciones de la variable **edad del pasajero (age)**?

```
df.titanic['age'].unique()
```

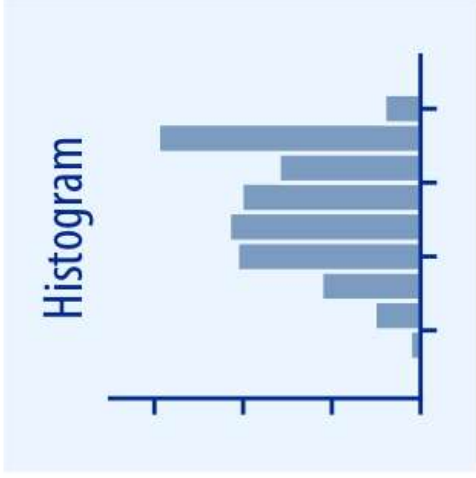
array([22.	38.	26.	35.	nan,	54.	2.	27.	14.
4.	58.	20.	39.	55.	31.	34.	15.	28.
8.	19.	40.	66.	42.	21.	18.	3.	7.
49.	29.	65.	28.5	5.	11.	45.	17.	32.
16.	25.	0.83,	30.	33.	23.	24.	46.	59.
71.	37.	47.	14.5	70.5	32.5	12.	9.	36.5
51.	55.5	40.5	44.	1.	61.	56.	50.	36.
45.5	20.5	62.	41.	52.	63.	23.5	0.92,	43.
60.	10.	64.	13.	48.	0.75,	53.	57.	80.

DATASET TITANIC - Variable **age**

En este caso se trata de una variable cuantitativa que **puede asumir un número relativamente grande de valores distintos.**

Si intentáramos representar su distribución con un gráfico de bastones, obtendríamos un bastón por cada edad registrada: el gráfico sería difícil de leer y perdería toda utilidad como herramienta de resumen.

HISTOGRAMA DE FRECUENCIAS



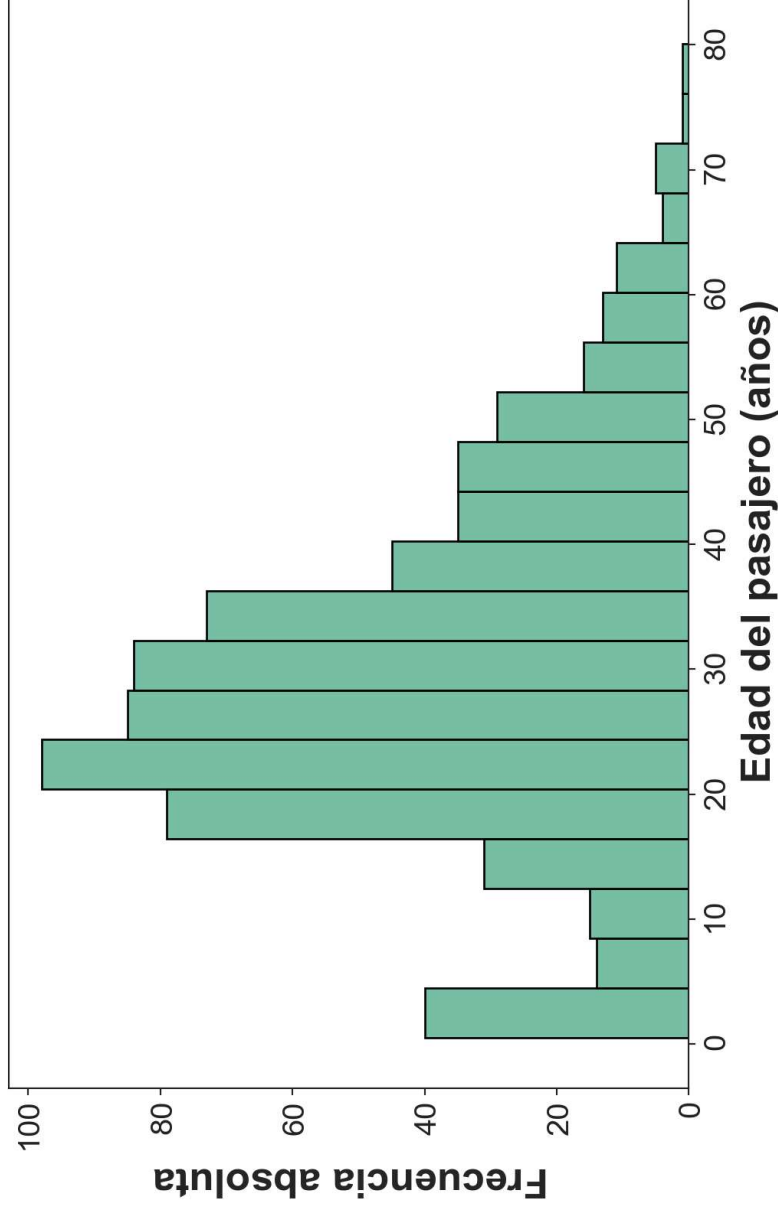
Un gráfico muy utilizado para representar datos de una variable cuantitativa que asume un gran número de valores diferentes es el **histograma de frecuencias**.

En el **eje de las abscisas** se representan **intervalos** en que se agrupan los valores de la variable y en el **eje de las ordenadas**, la **frecuencia absoluta o relativa**. Luego, sobre cada uno de los subintervalos se grafica **un rectángulo** cuya **área** representa la frecuencia (absoluta o relativa) del mismo.

HISTOGRAMA DE FRECUENCIAS

Podemos construir este tipo de visualización utilizando la función **histplot()** de la librería **seaborn**. Veamos cómo luce el histograma de las observaciones de *edad del pasajero*:

```
sns.histplot(x = 'age', data = df_titanic);
```

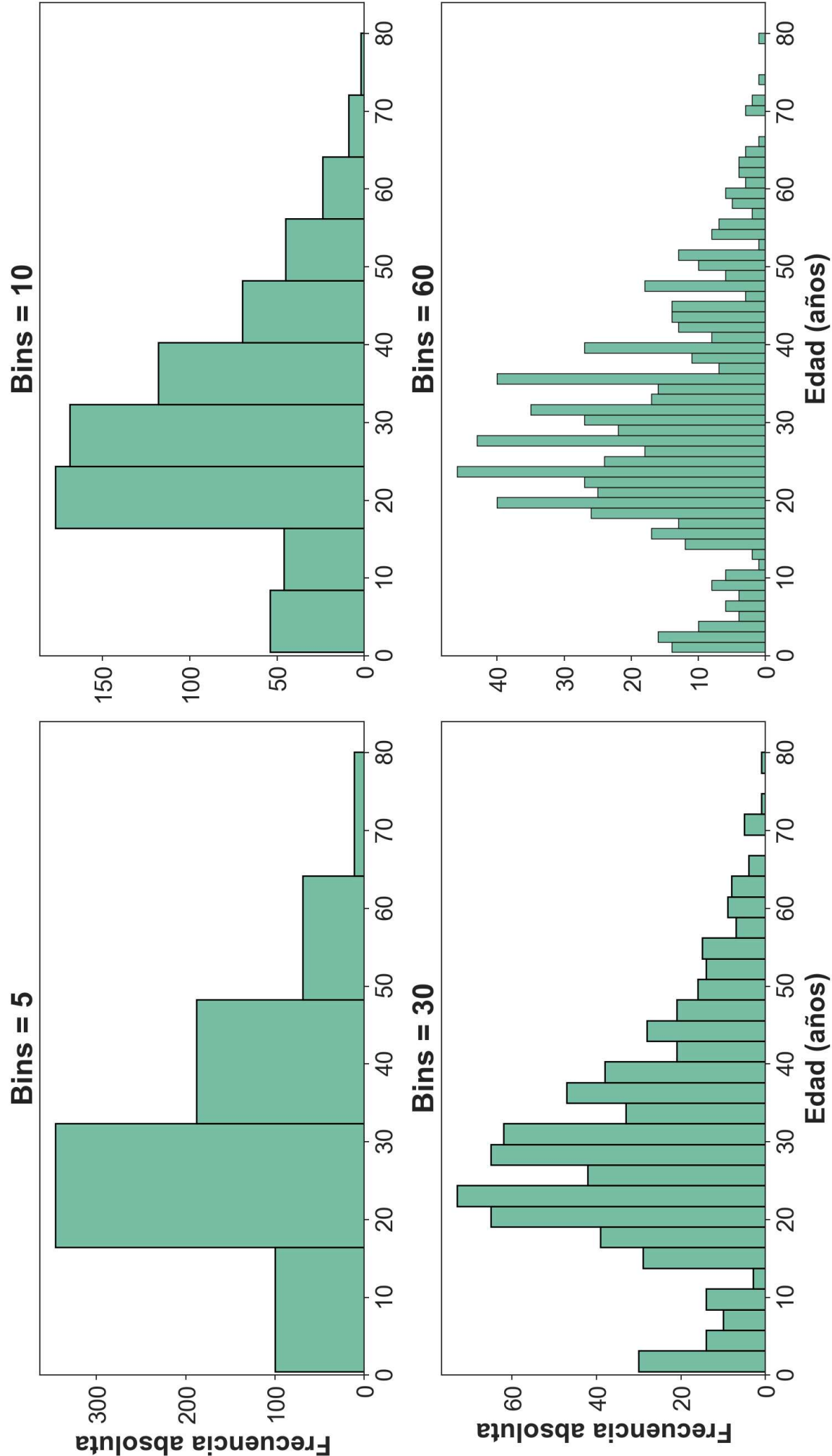


HISTOGRAMA DE FRECUENCIAS

Por *default*, el número de subintervalos en los que se segmenta la variable (**bins**) es calculado por la función de **seaborn** utilizando algún método predeterminado que toma en cuenta la cantidad de observaciones que se quieren representar.

¿Qué efecto produce la modificación del número de bins sobre el gráfico resultante?

HISTOGRAMA DE FRECUENCIAS



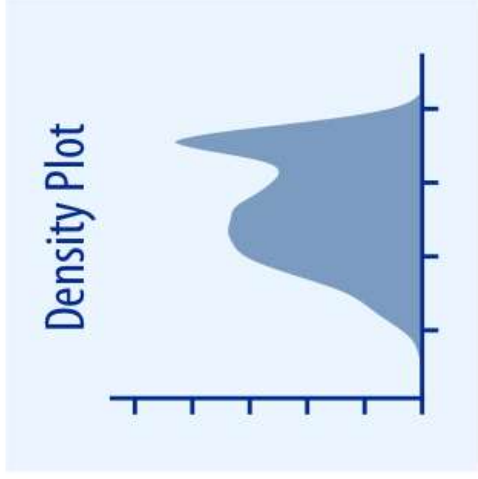
HISTOGRAMA DE FRECUENCIAS

El parámetro **bins** de **sns.histplot()** puede estar dado por:

- el nombre de algún método o regla de referencia: por ejemplo, **bins = 'sqrt'**, para que utilice el criterio de la raíz cuadrada de n para el número de bins.
- el número de bins: por ejemplo, **bins = 10**.
- una lista con los *breaks* que definen los subintervalos: por ejemplo, **bins = [0, 5, 10, 15, 20]**.

Otros parámetros que puede ser útil setear son **binwidth** (amplitud de cada intervalo) y **binrange** (rango cubierto por el histograma).

GRÁFICO DE DENSIDAD



Un **gráfico de densidad** intenta visualizar la **distribución de probabilidad** subyacente de los datos mediante el dibujo de una curva continua apropiada, la cual debe ser estimada a partir de los datos.

El método de estimación más comúnmente utilizado se conoce como **estimación de densidad por kernel**.

ESTIMACIÓN DE DENSIDAD POR KERNEL

En la estimación de densidad por kernel se “dibuja” una curva suave llamada **kernel** con un ancho determinado (controlado por un parámetro llamado **ancho de banda**) en la ubicación de cada punto de datos, y luego se suman todas estas curvas para obtener la estimación global de la densidad.

El *kernel* más utilizado es el gaussiano, pero hay muchas otras opciones.

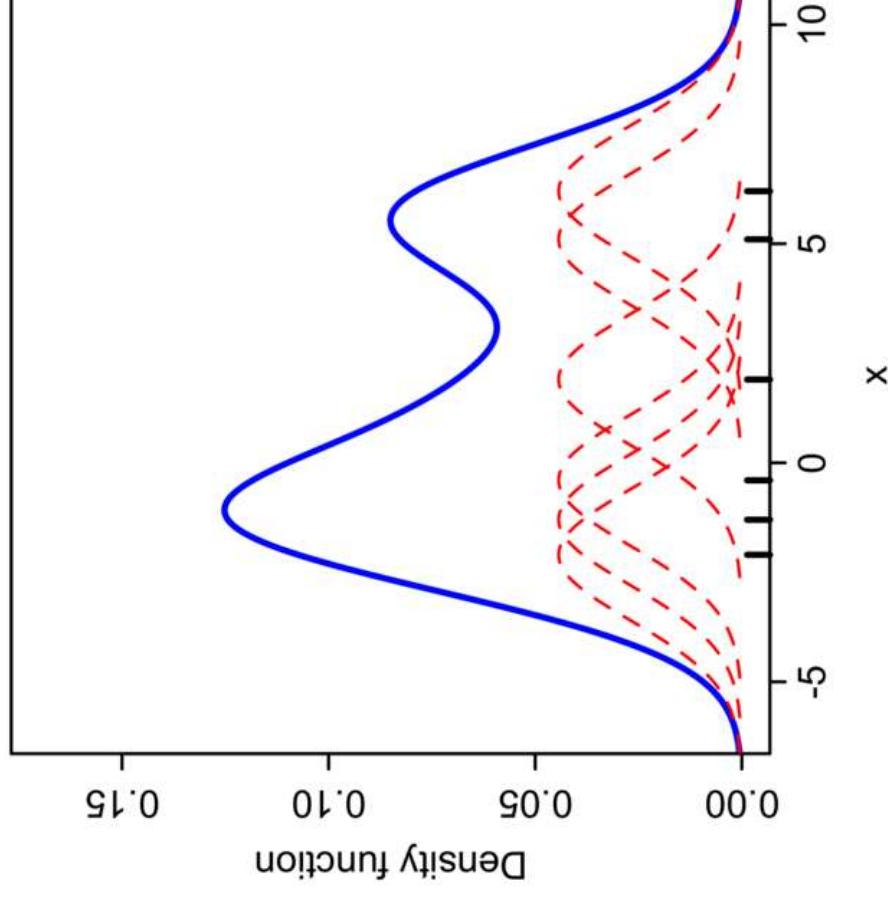


GRÁFICO DE DENSIDAD

Para hacer un gráfico de densidad a partir de las observaciones de la variable *edad del pasajero* podemos utilizar la función **kdeplot()** de **seaborn**:

```
sns.kdeplot(x = 'age', fill = True, data = df_titanic);
```

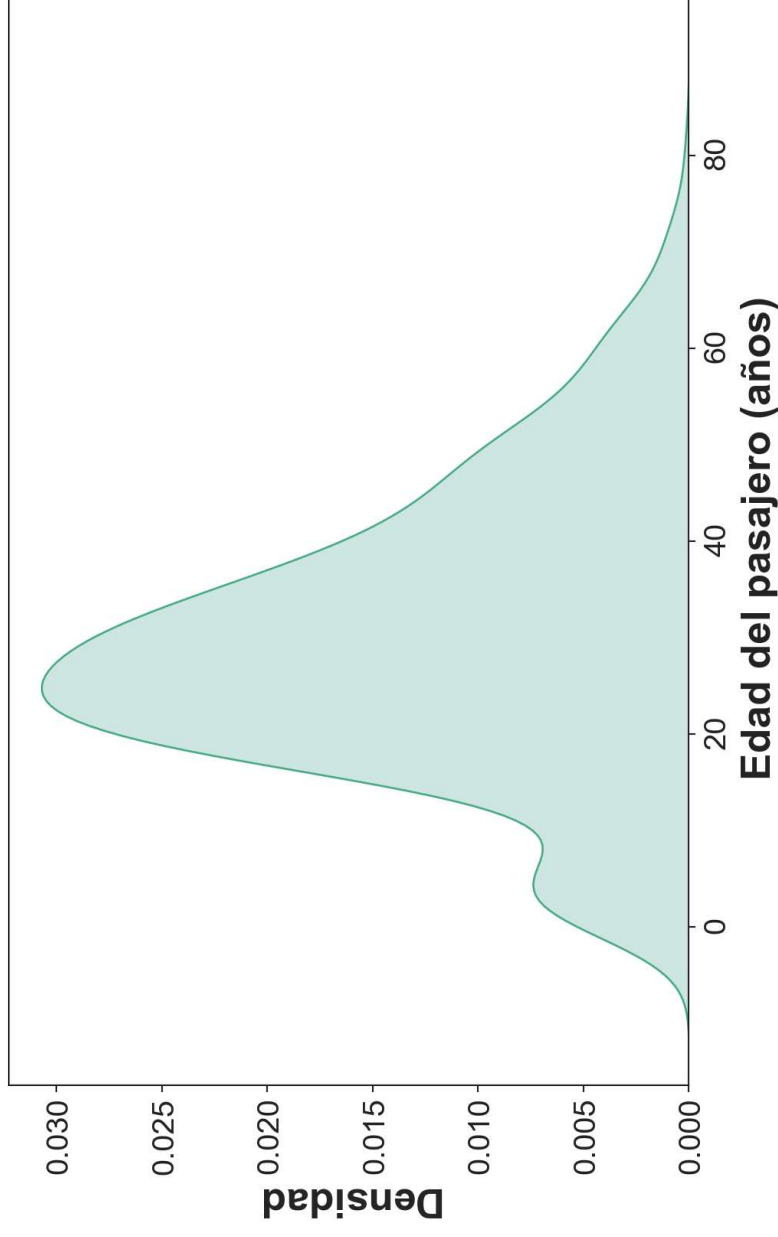
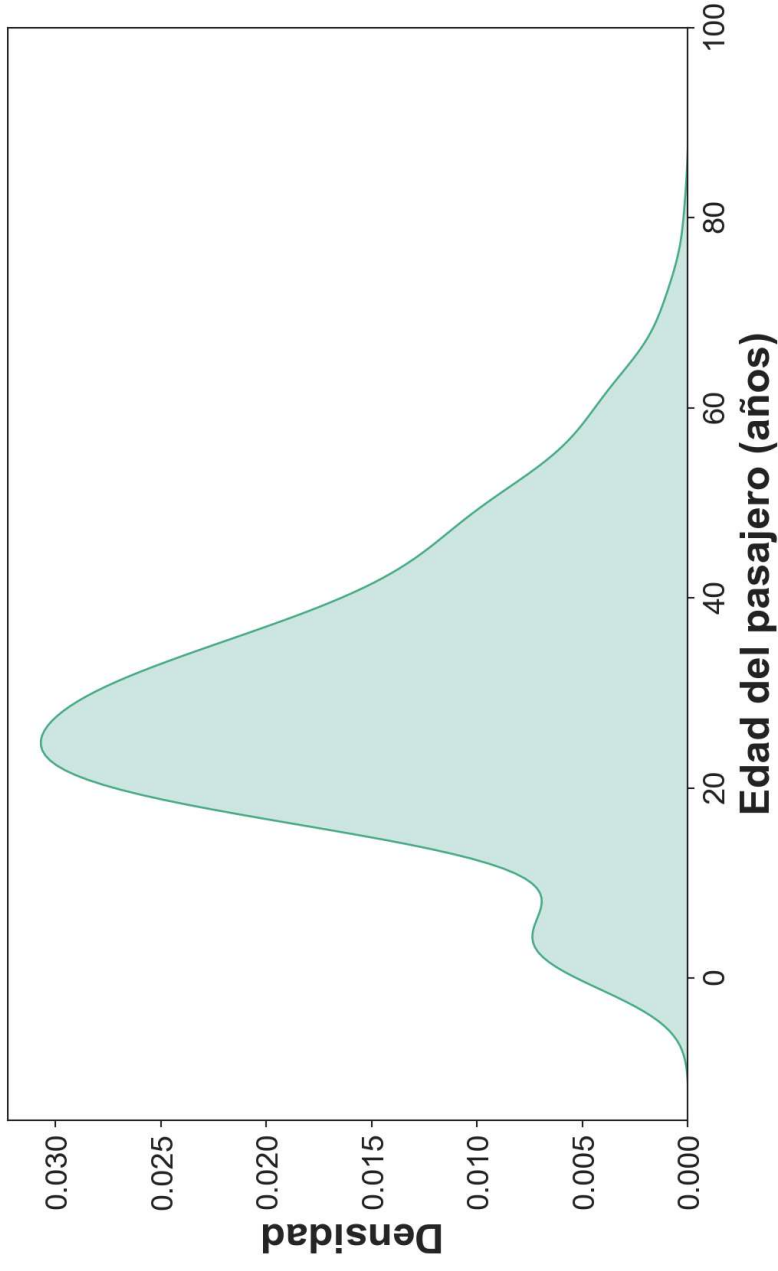


GRÁFICO DE DENSIDAD



🙄 ¿Qué se observa en el gráfico? ¿Hay algo que podamos *criticar*?

GRÁFICO DE DENSIDAD

La trampa de los gráficos de densidad.

🧐 Las estimaciones de densidad por kernel tienen una trampa de la que debemos ser conscientes: **tienden a producir la apariencia de que hay datos donde no existen, en particular en las colas.**

🧐 Para evitar que el gráfico se extienda hacia valores que resultan inconsistentes para el tipo de variable con el que trabajamos (por ejemplo, valores negativos de la edad de los pasajeros), podemos setear el parámetro **clip** de manera de “cortar” el gráfico de densidad en puntos específicos del eje.

GRÁFICO DE DENSIDAD ⚠️

```
sns.kdeplot(x = 'age', fill = True, data = df_titanic);
```

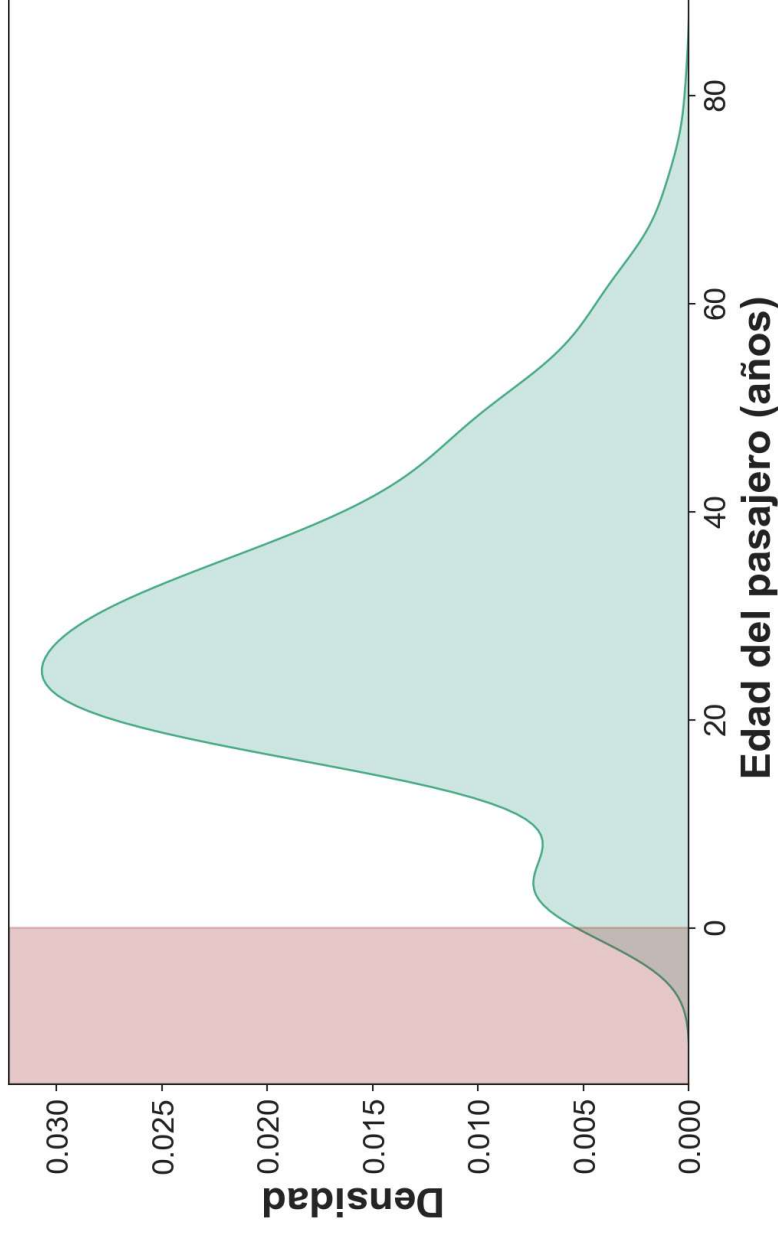


GRÁFICO DE DENSIDAD 🤪

```
sns.kdeplot(x = 'age', clip = [0,100], fill = True, data = df_titanic);
```

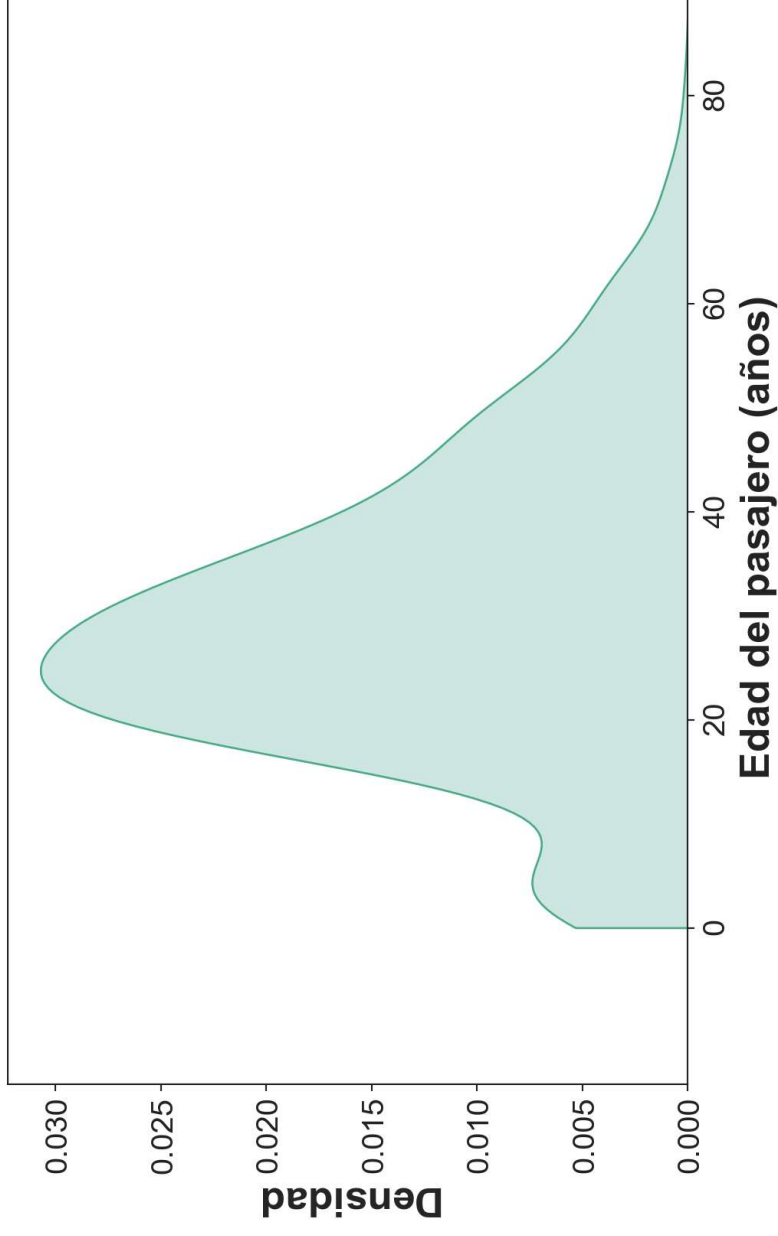


GRÁFICO DE DENSIDAD 🧐

```
sns.kdeplot(x = 'age', clip = [0,100], fill = True, data = df_titanic)
plt.xlim(0, 85);
```

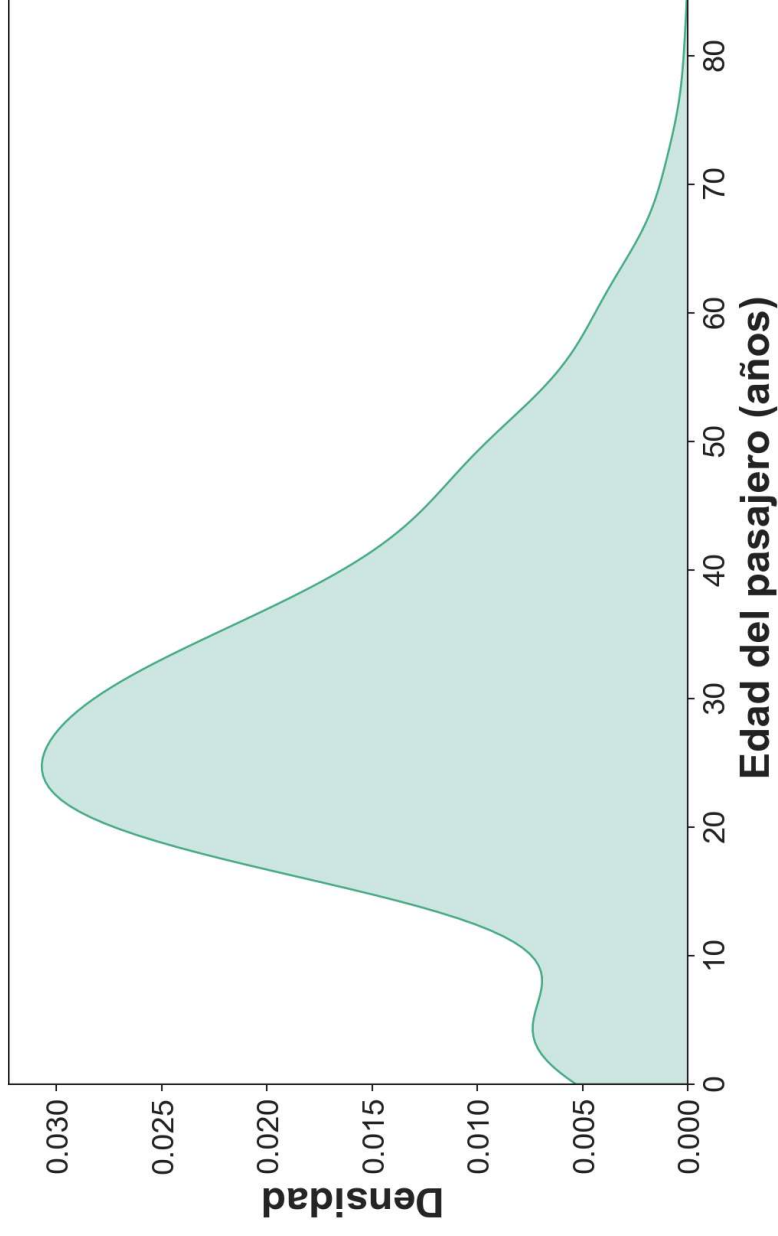


Gráfico con límites en el eje x ajustados

GRÁFICO DE DENSIDAD

Sobre la escala del eje “Densidad”.

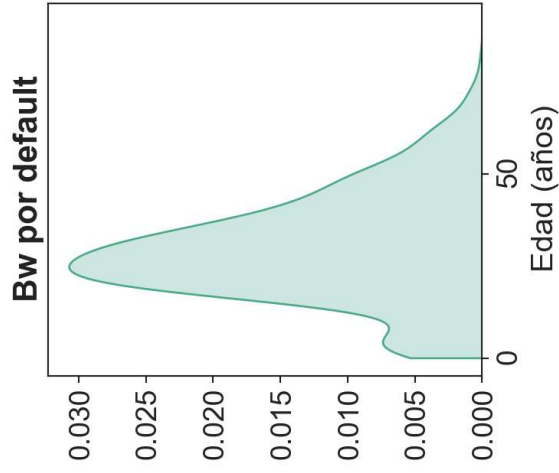
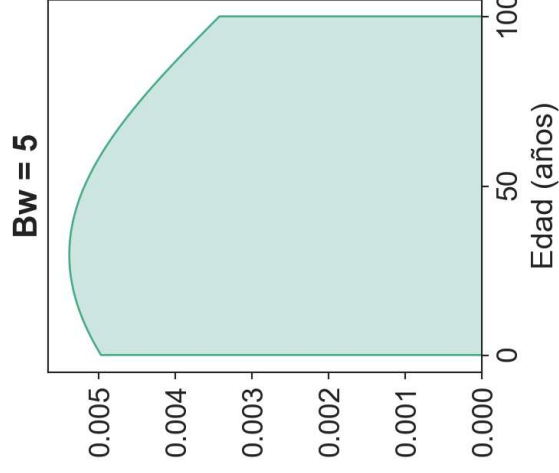
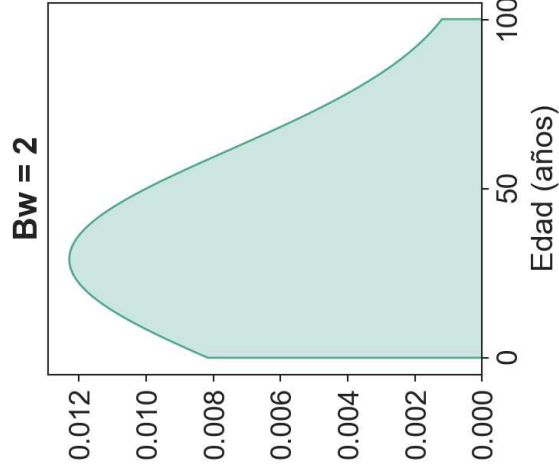
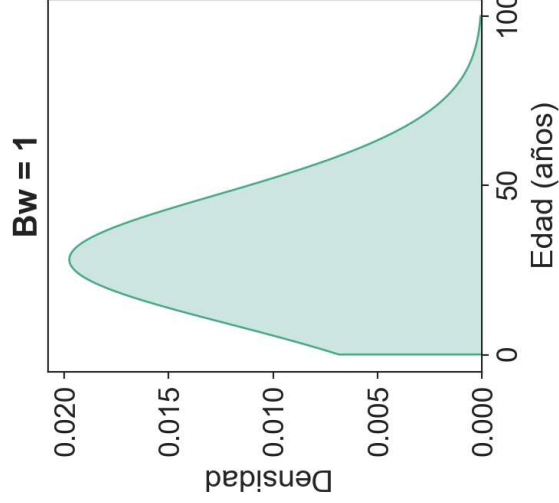
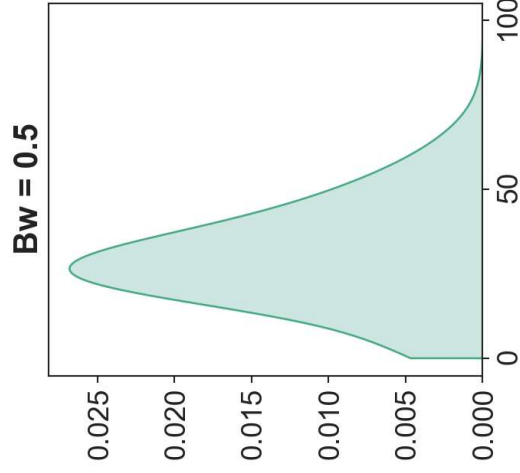
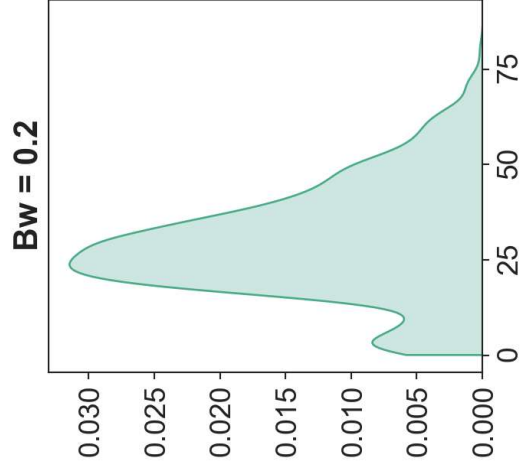
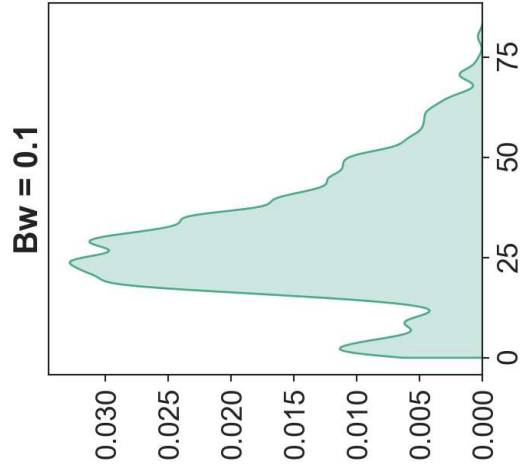
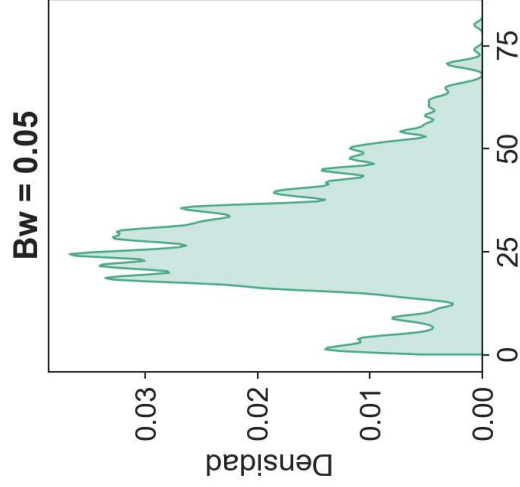
Como estimaciones de las distribuciones de probabilidad de variables continuas, las curvas de densidad suelen escalarse de manera que **el área bajo la curva sea igual a uno**. Esta convención puede hacer que la escala del eje y sea confusa, ya que depende de las unidades del eje x .

GRÁFICO DE DENSIDAD

La apariencia visual exacta de un gráfico de densidad dependerá tanto de la elección del ancho de banda como del tipo de kernel. La primera característica puede modificarse dentro del parámetro **bw_method** de **sns.kdeplot()**, el cual puede ser:

- el nombre de un método de estimación del *bandwidth* según ciertos criterios: por ejemplo, **bw_method = 'scott'** (*default*).
- un valor de ancho de banda específico: por ejemplo: **bw_method = 0.5**.

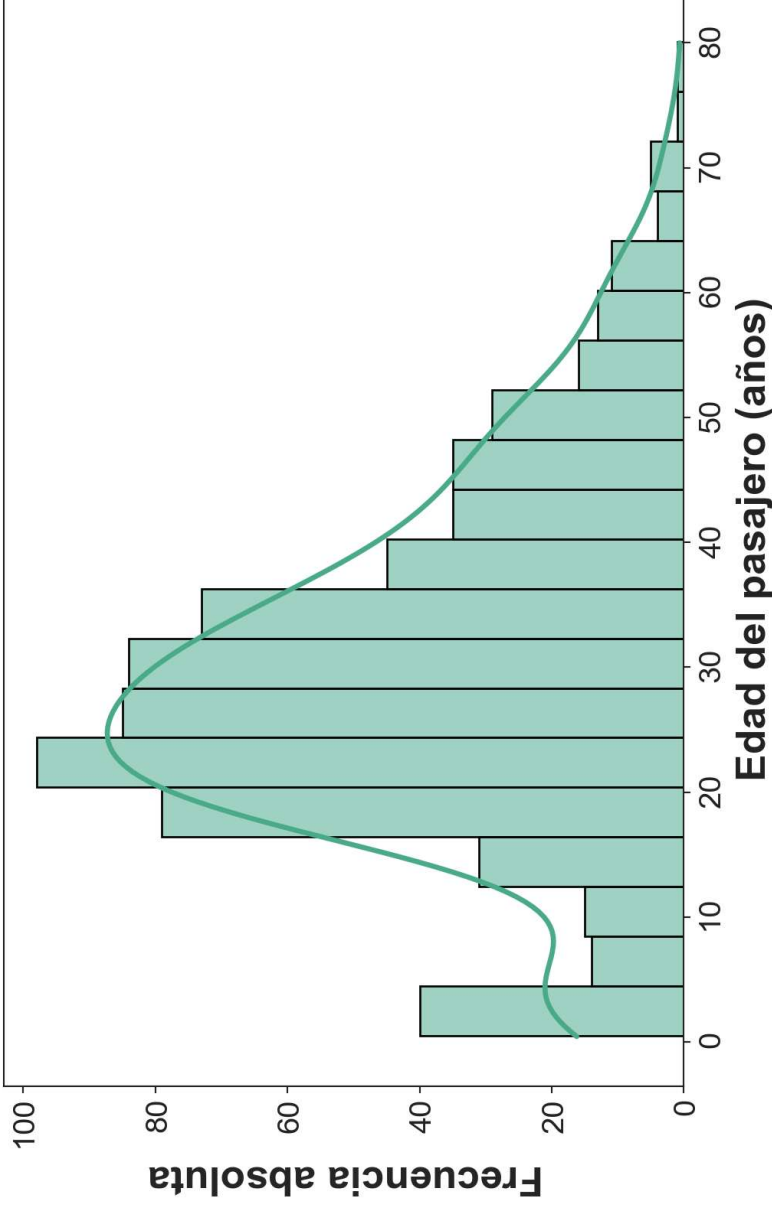
¿Qué efecto produce la modificación del ancho de banda (bandwidth) sobre el gráfico resultante?



HISTOGRAMA + GRÁFICO DE DENSIDAD

Seteando el parámetro **kde = True** de **sns.histplot()** podemos superponer la curva de densidad por Kernel al histograma:

```
sns.histplot(x = 'age', kde = True, data = df_titanic);
```



COMPARAR DISTRIBUCIONES ENTRE GRUPOS

Hasta aquí visualizamos la distribución de **una variable cuantitativa** de forma global. Sin embargo, en muchos análisis el interés está en **comparar cómo se distribuye una variable cuantitativa en distintos grupos** definidos por una **variable categórica**.

Para ilustrar las distintas alternativas, retomamos el dataset **Palmer Penguins** y la variable **flipper_length_mm**:

```
df_penguins = sns.load_dataset('penguins')
```

COMPARAR DISTRIBUCIONES ENTRE GRUPOS

A continuación, vamos a presentar los siguientes gráficos:

- **Gráficos de densidad múltiples**
- **Boxplot múltiple**
- **Strip plot**
- **Violin plot**
- **Ridgeline plot**

GRÁFICOS DE DENSIDAD MÚLTIPLES

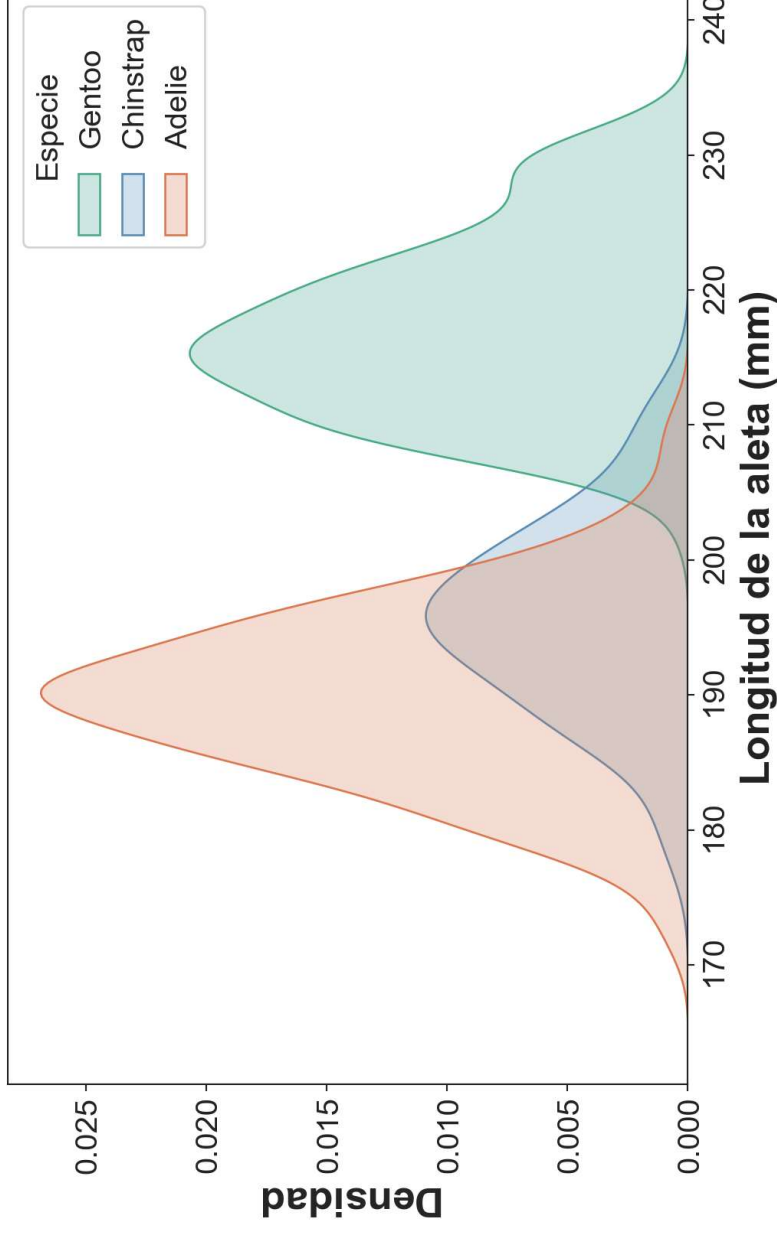
Si quisiéramos visualizar la distribución de la *longitud de la aleta* de los pingüinos no en forma general sino segmentada según *especie* podemos setear los siguientes parámetros dentro de

`sns.kdeplot()`:

- **`hue = 'species'`** , para representar con un color distinto la distribución de la variable para cada especie (Gentoo/Adelie/Chinstrap).
- **`multiple = 'layer'`** , para superponer las distintas curvas.

GRÁFICOS DE DENSIDAD MÚLTIPLES

```
sns.kdeplot(x = 'flipper_length_mm', hue = 'species', multiple = 'layer',
```



 ¿Qué especie presenta, en promedio, mayores longitudes de aleta?

 ¿Qué especies son más similares entre sí respecto a sus longitudes de aleta?

  A partir de este gráfico, ¿es posible identificar de qué especie hay menos pingüinos en el dataset? ¿Por qué?

GRÁFICOS DE DENSIDAD MÚLTIPLES

Porcentaje que representa cada especie sobre el total:

```
df_penguins['species'].value_counts(normalize = True).mul(100).round(2)
```

species

Adelie 44.19

Gentoo 36.05

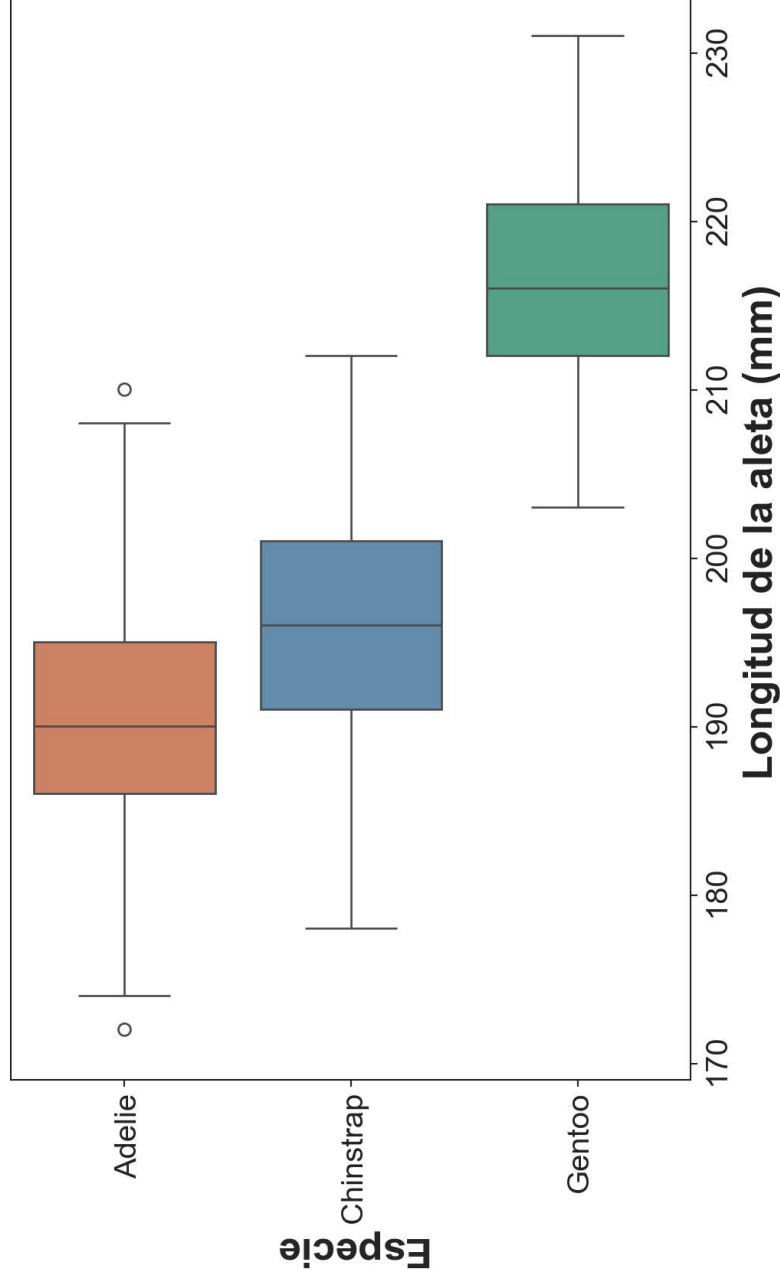
Chinstrap 19.77

Name: proportion, dtype: float64

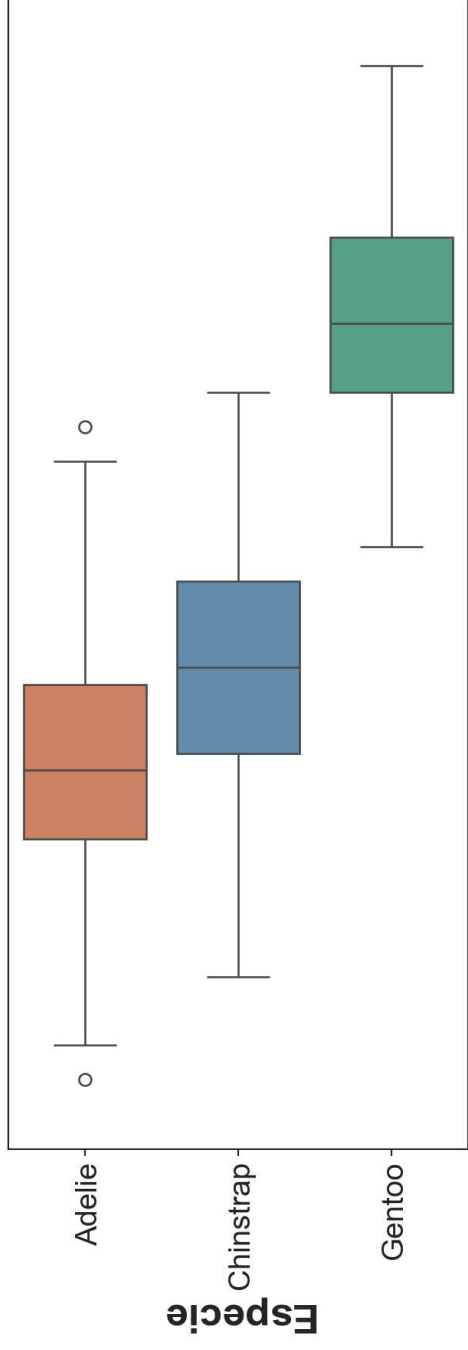
BOXPLOT MÚLTIPLE

Un **boxplot múltiple** es otro gráfico que permite visualizar si existen diferencias en la longitud de la aleta según la especie:

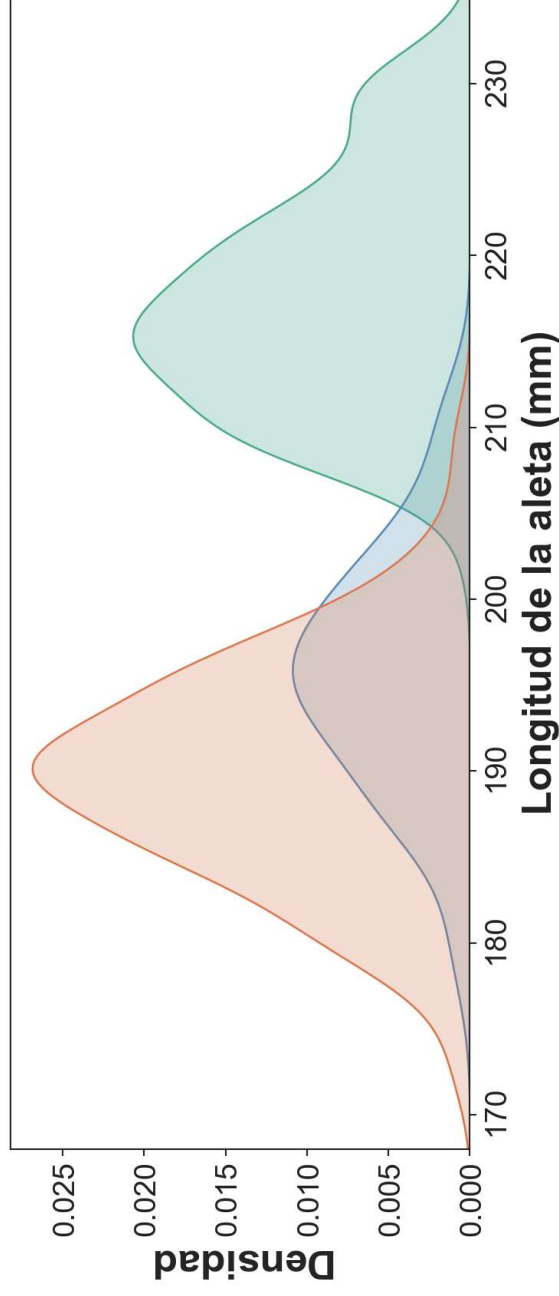
```
sns.boxplot(x = 'flipper_length_mm', y = 'species', hue = 'species', data =
```



BOXPLOT MÚLTIPLE



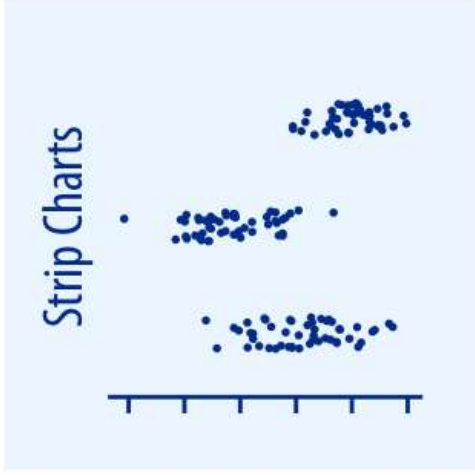
GRÁF. DE DENSIDAD MÚLTIPLE



¿Qué información resume este gráfico sobre la distribución de la longitud de la aleta en cada especie?

En comparación con el gráfico de densidad, ¿qué aspectos de la distribución se pierden o no pueden observarse aquí?

STRIP PLOTS o JITTER PLOTS



Los **jitter plots** son gráficos en los que **se representan directamente todos los datos individuales en la forma de puntos**.

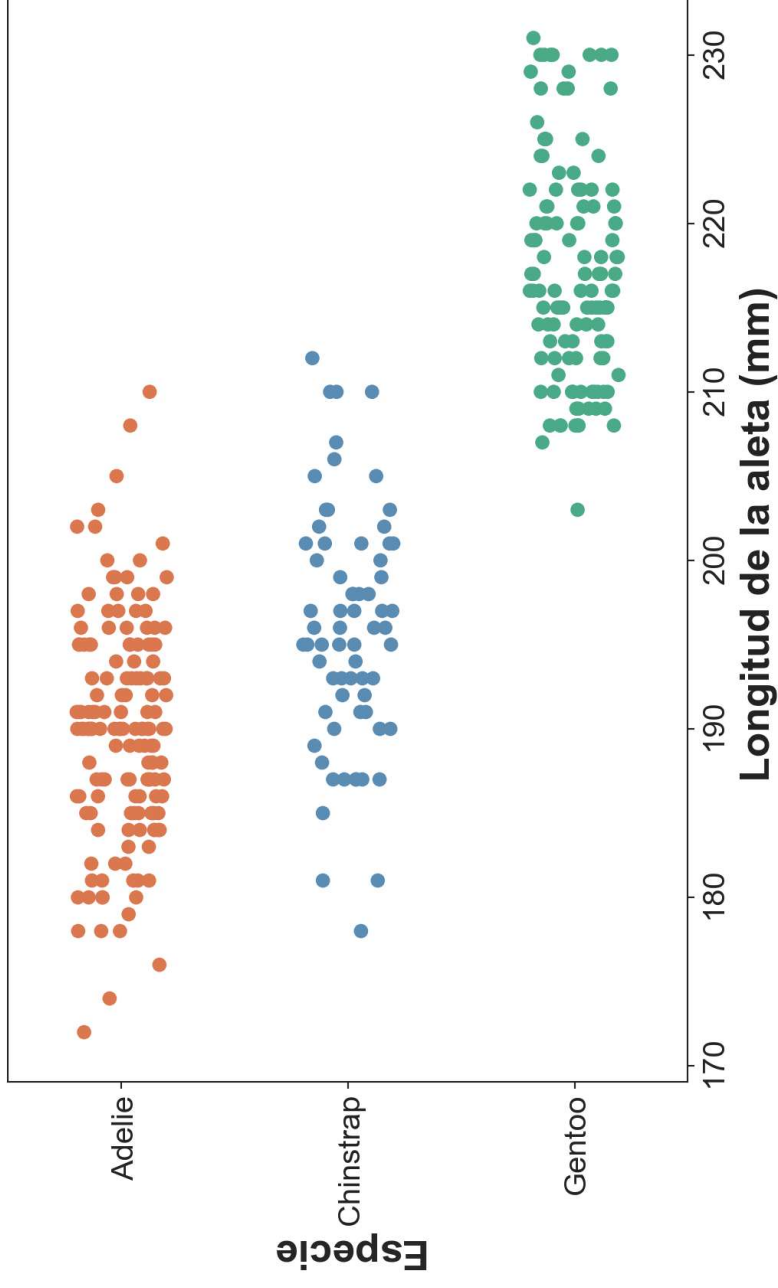
Cuando hay muchos datos, para no graficar demasiados puntos uno encima del otro, los mismos se dispersan un poco añadiendo algo de ruido aleatorio en la dimensión en la que se plotean (técnica llamada **“jittering”**).

Constituyen una buena herramienta para visualizar múltiples distribuciones.

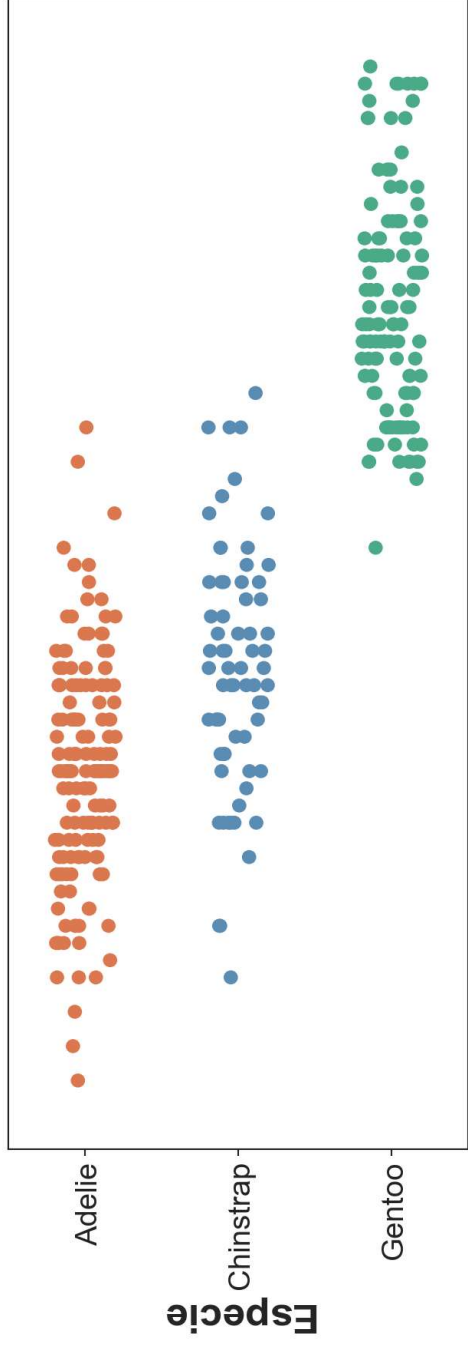
STRIP PLOTS o JITTER PLOTS

Podemos construirlo utilizando la función **stripplot()** de **seaborn**. El grado de “jittering” se setea a través del parámetro **jitter**.

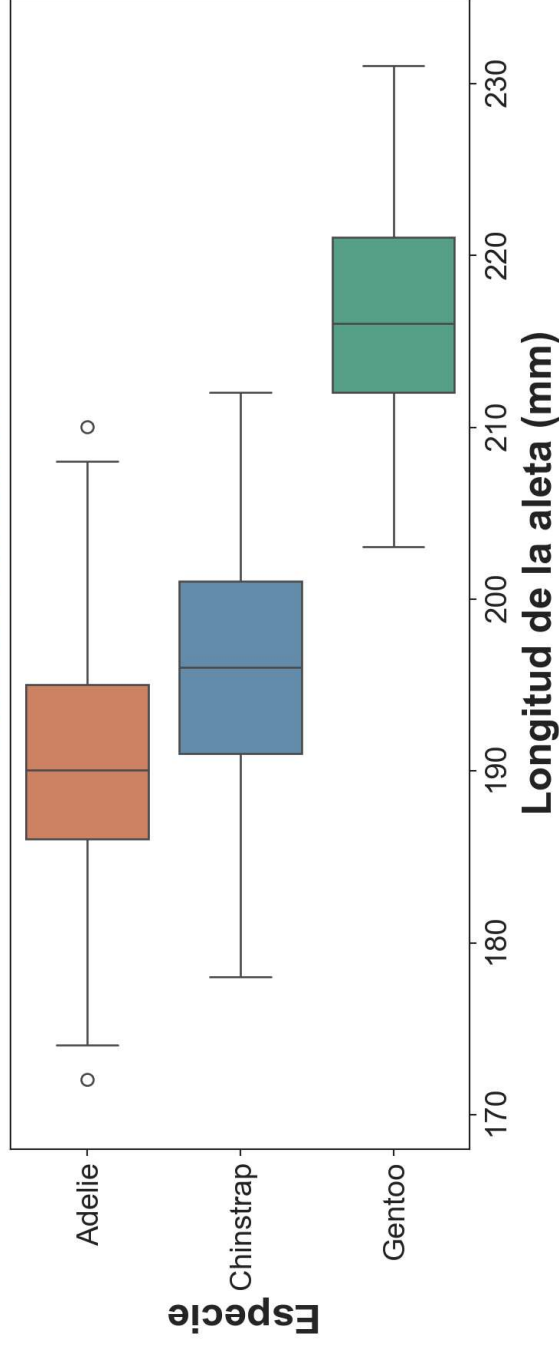
```
sns.stripplot(x = 'flipper_length_mm', y = 'species', hue = 'species', jitter
```



STRIP PLOT



BOXPLOT MÚLTIPLE

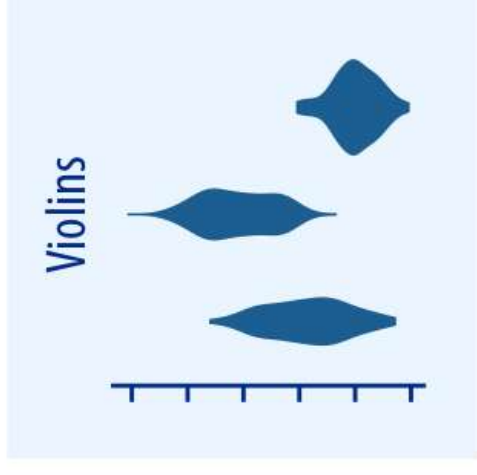


En comparación con el boxplot múltiple, ¿qué información adicional aporta este gráfico?

¿En qué situaciones preferiríamos usar un strip plot en lugar de un boxplot?

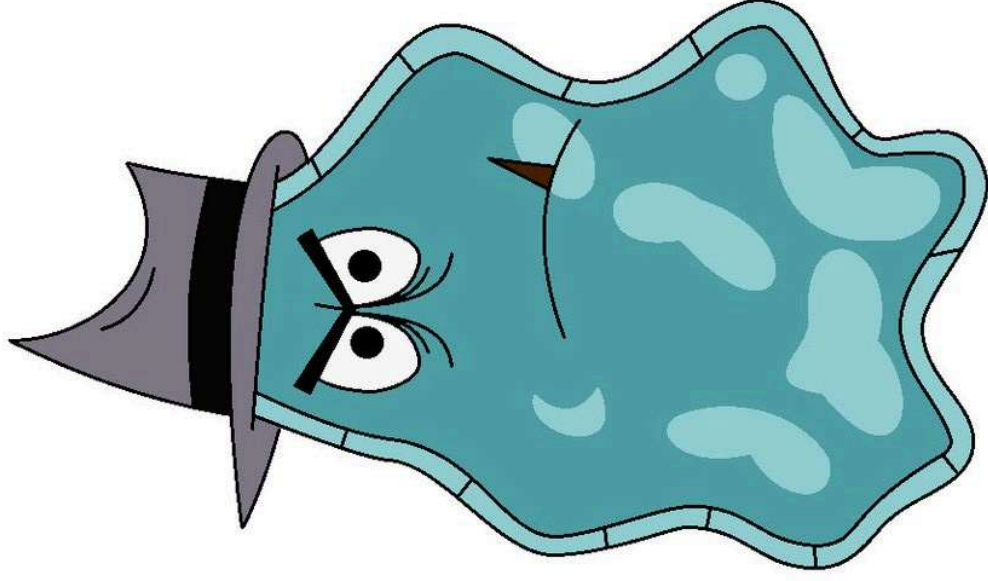
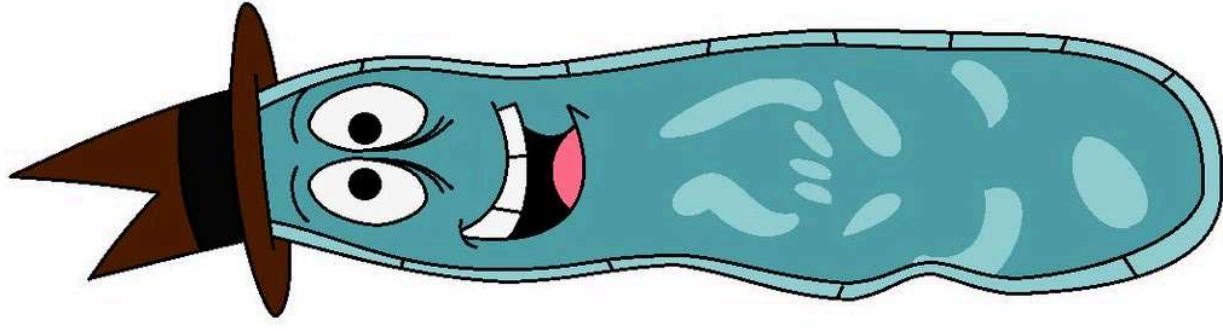
VIOLIN PLOT

Los **gráficos de violín** (*violin plots*) representan una **estimación de densidad por Kernel**, mostrada de **forma espejada**, generando una figura simétrica cuyo ancho en cada valor es proporcional a la **densidad de observaciones**.



Limitación: requiere un número suficiente de observaciones por grupo; de lo contrario, la estimación de densidad puede ser inestable y sugerir estructuras (como múltiples picos) que no reflejan características reales de la distribución

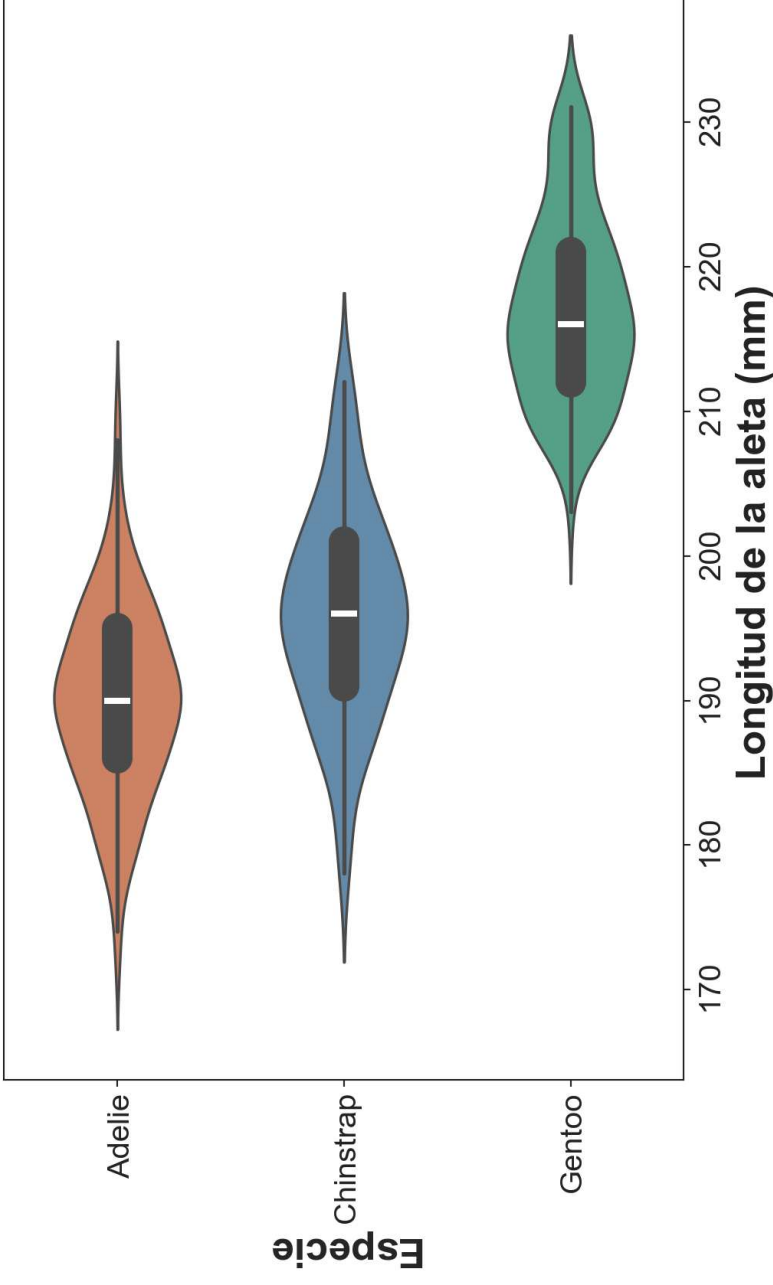
Cualquier parecido con *la ficción* es pura coincidencia...



VIOLIN PLOT

Podemos construir uno con los datos de *long. de la aleta* según *especie* utilizando la función **violinplot()** de **seaborn**.

```
sns.violinplot(x = 'flipper_length_mm', y = 'species', hue = 'species', ga
```



VIOLIN PLOT

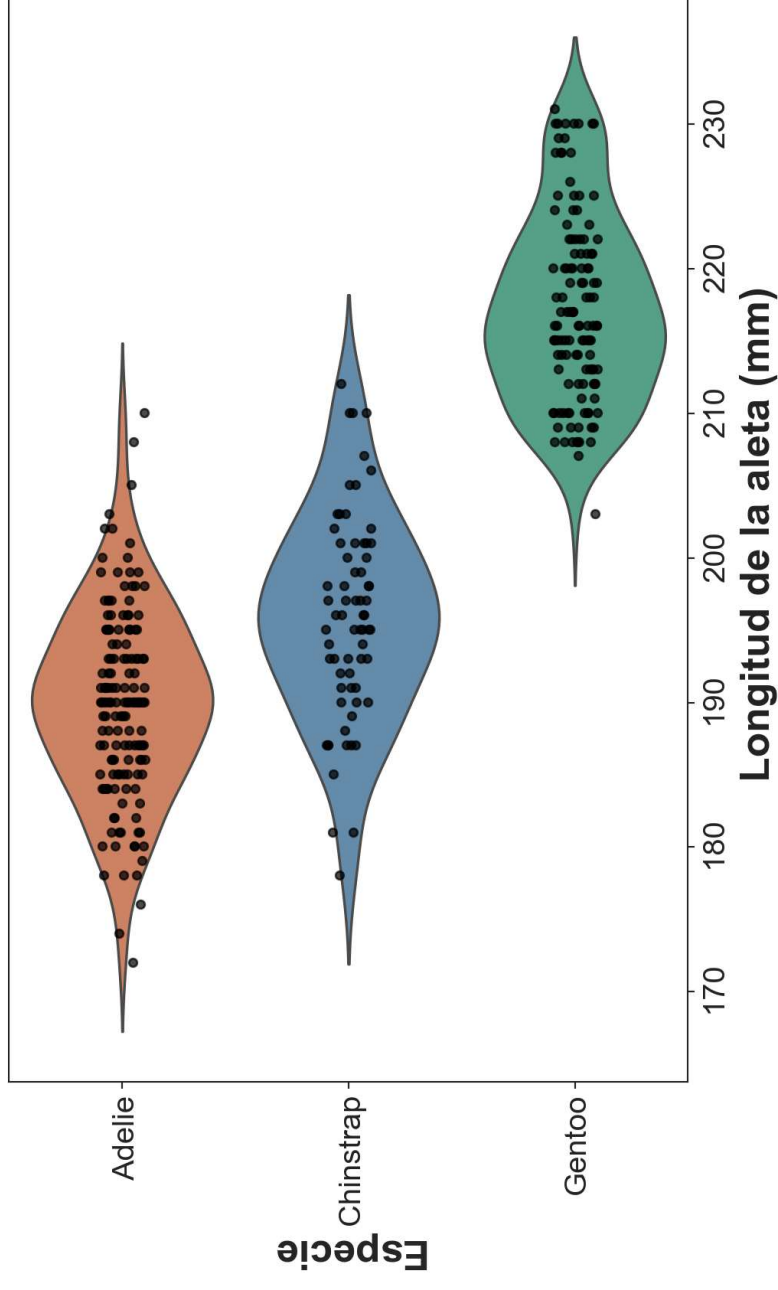
Por default, la visualización incorpora un **boxplot interno** que aporta algunas medidas resumen, como la mediana y el rango intercuartílico. Si se desea, puede ocultarse seteando el parámetro **inner = None** o “tunearse” introduciendo modificaciones en la forma de un diccionario dentro de **inner_kws**.

Por otra parte, otro parámetro que permite modificar la apariencia de la visualización es **gap**, para introducir cambios en el grosor del gráfico de violín.

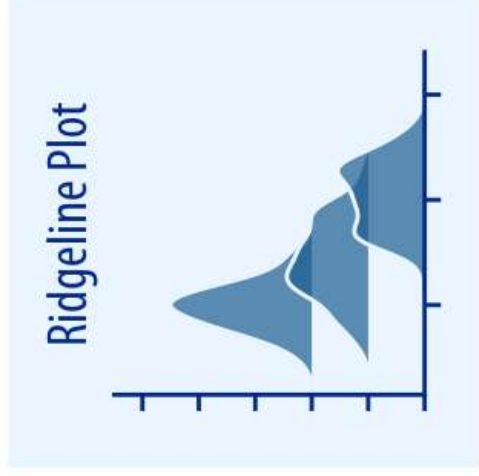
STRIP PLOT + VIOLIN PLOT (SINA PLOT)

Los dos últimos gráficos pueden combinarse para crear una visualización muy interesante, llamada **sina plot**

```
sns.violinplot(x = 'flipper_length_mm', y = 'species', hue = 'species', data = df_penguins,  
sns.stripplot(x = 'flipper_length_mm', y = 'species', data = df_penguins,
```



RIDGELINE PLOT



Los **ridgeline plots** representan **múltiples distribuciones** (generalmente mediante densidades por Kernel) desplazadas verticalmente. Cada curva muestra la distribución de la variable para un grupo, y su superposición genera una visualización similar a una “cadena montañosa”.

RIDGELINE PLOT

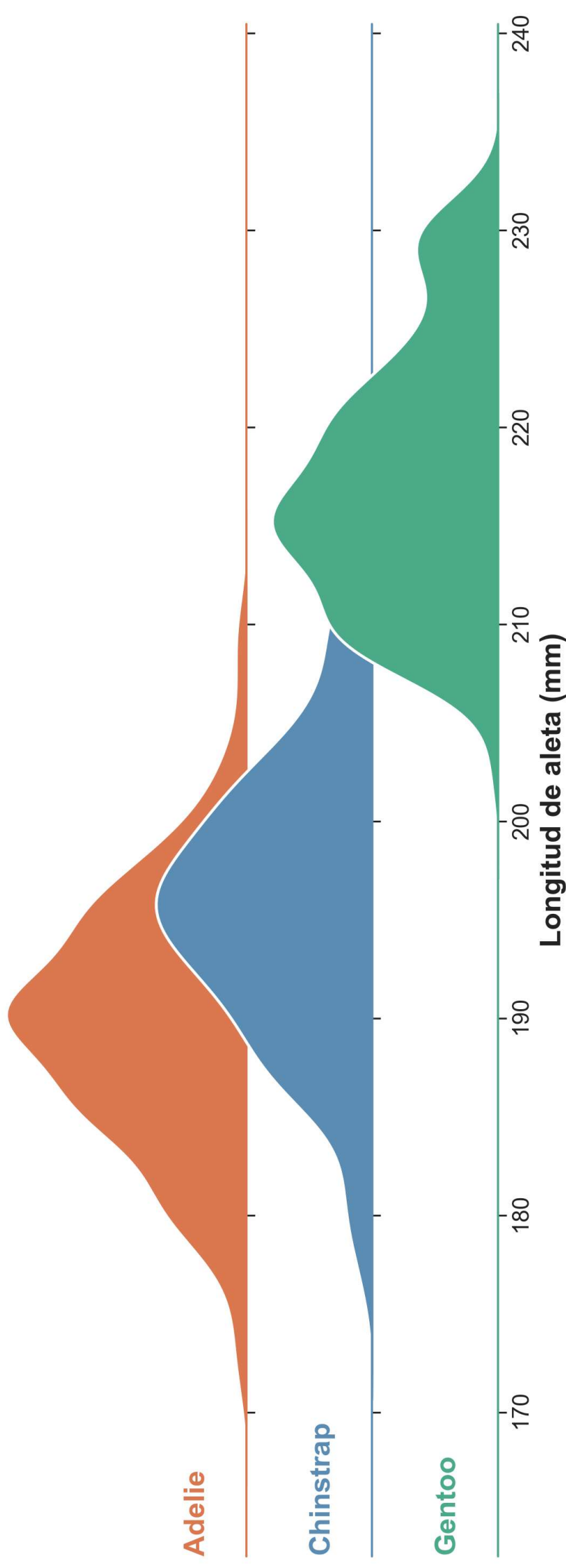
Son especialmente útiles para:

- **Comparar la forma y la posición de las distribuciones** entre muchos grupos, ya sean categorías nominales o grupos con un orden natural
- **Visualizar cambios en las distribuciones a lo largo del tiempo**, cuando cada grupo representa un periodo de tiempo determinado.

Limitación: cuando hay mucho solapamiento, pocas observaciones por grupo, o un número muy elevado de grupos, la interpretación puede volverse menos clara.

RIDGELINE PLOT

Volviendo al ejemplo en el que analizamos la longitud de la aleta según la especie:

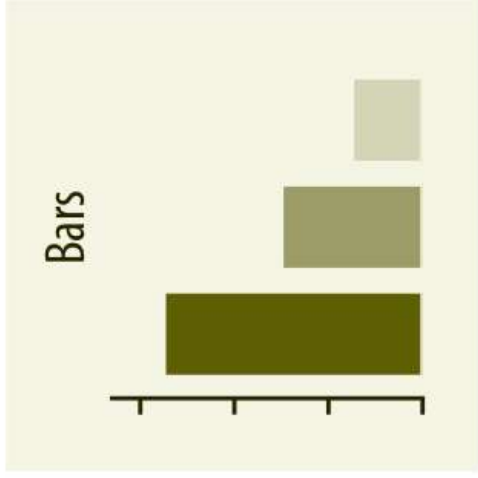


GRÁFICOS PARA VISUALIZAR CONTEOS/PROPORCIO NES

GRÁFICOS PARA CONTEOS/PROPORCIONES

Hasta aquí trabajamos principalmente con variables cuantitativas. Cuando la variable de interés es **cualitativa**, el objetivo cambia: ya no buscamos describir una forma continua, sino **comparar frecuencias o proporciones entre categorías**.

GRÁFICOS DE BARRAS



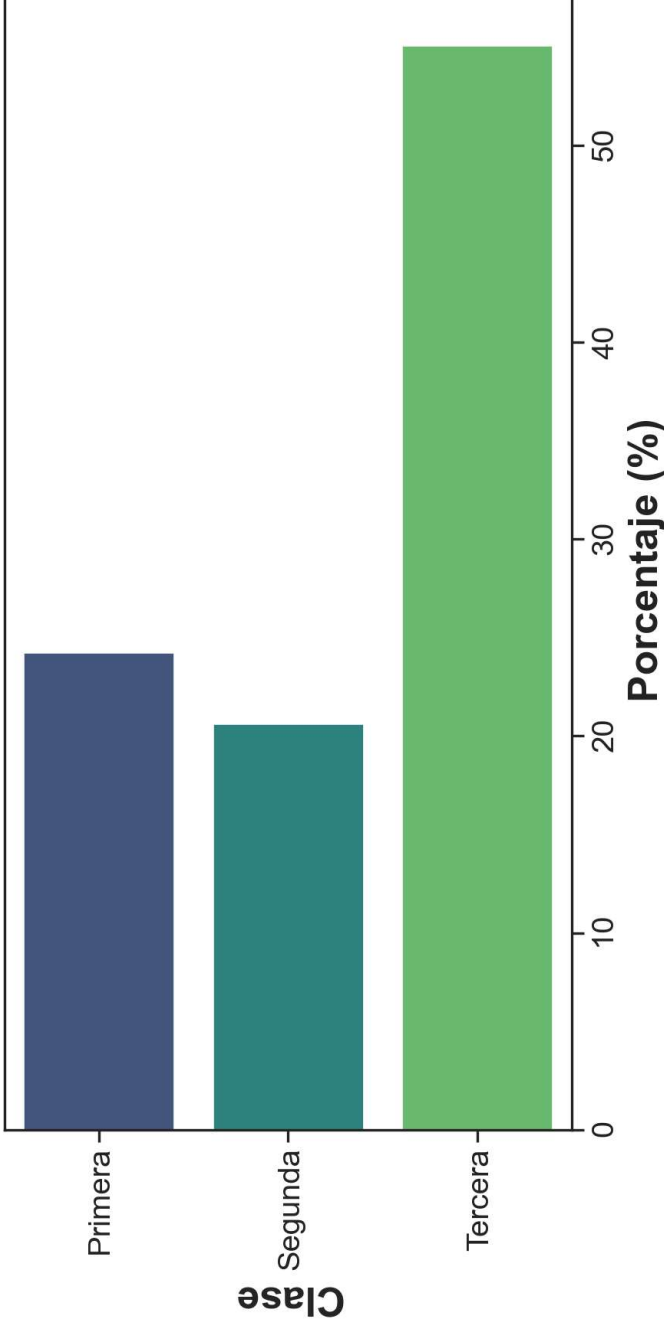
Uno de los gráficos más utilizados para representar datos de variables cualitativas son los **gráficos de barras**.

En este tipo de gráficos, se representa **una barra (por lo general, horizontal) para cada categoría**. La **longitud** de cada una de ellas es **proporcional al porcentaje de unidades que pertenecen a la categoría** y el **ancho es el mismo** para todas.

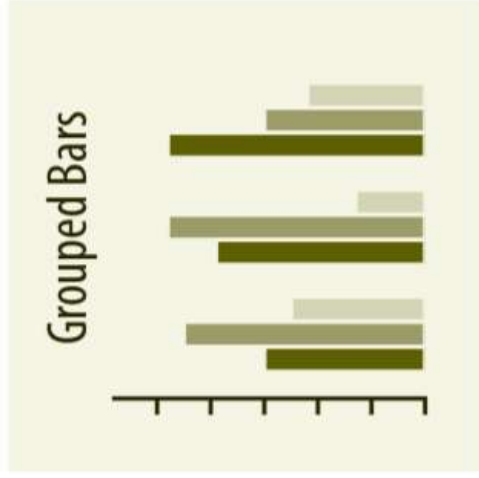
GRÁFICOS DE BARRAS

Podemos representar la distribución de pasajeros que viajaron en cada **clase** (**class**) utilizando un gráfico de barras:

```
sns.countplot(y = 'class', stat = 'percent', data = df_titanic);
```



BARRAS AGRUPADAS O PARALELAS



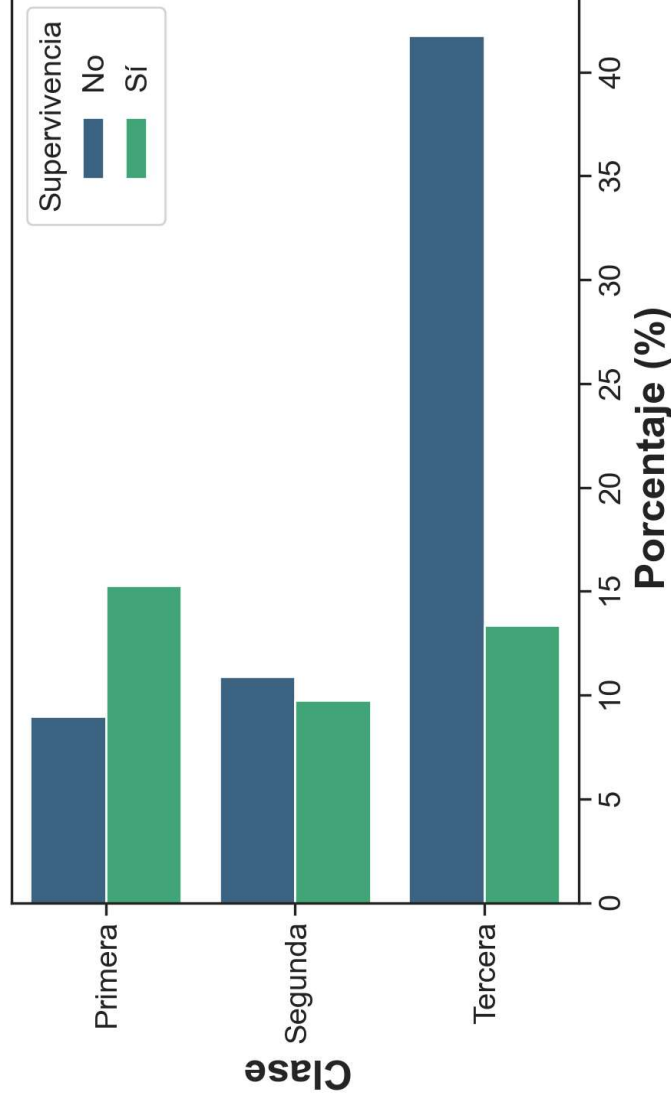
A menudo estamos interesados en visualizar **dos variables categóricas al mismo tiempo**. Esto puede hacerse a través de un **gráfico de barras paralelas o agrupadas**.

En este tipo de gráficos, representamos un grupo de barras en cada posición a lo largo del eje *y*, de acuerdo a los distintos niveles de una primera variable categórica, y luego relacionamos cada una de las barras del grupo a un nivel diferente de una segunda variable categórica.

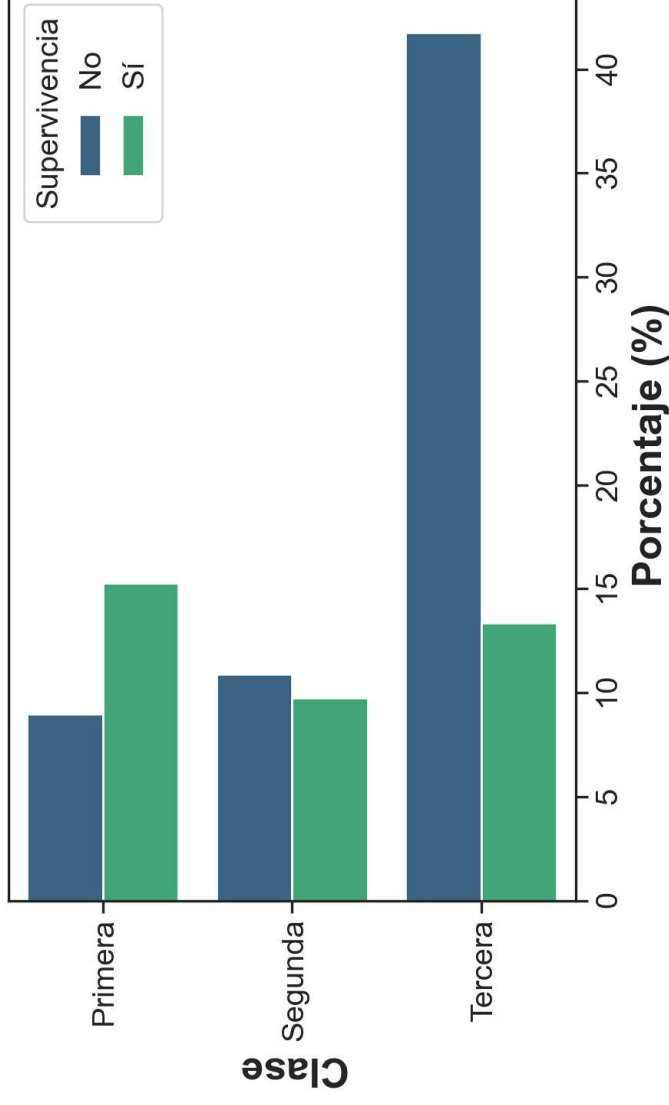
BARRAS AGRUPADAS O PARALELAS

Para construir un gráfico de barras paralelas que muestre los porcentajes de pasajeros según la **clase en la que viajaron** y **supervivencia al naufragio** (**alive**), podemos setear, en el gráfico anterior, el parámetro **hue = 'alive'**.

```
sns.countplot(y = 'class', hue = 'alive', stat = 'percent', data = df_tita
```



BARRAS AGRUPADAS O PARALELAS



🤔 ¿Qué conclusiones se pueden extraer a partir del gráfico?

BARRAS AGRUPADAS O PARALELAS

Los porcentajes de cada combinación de categorías están calculados **sobre el total de observaciones**. Podríamos acceder a esa información contando las ocurrencias de cada combinación de **class** y **alive**, y luego expresar esos valores como proporciones sobre el total:

```
df_titanic.value_counts(['class', 'alive'], normalize = True).mul(100).rou
```

class	alive	
First	no	8.98
	yes	15.26
Second	no	10.89
	yes	9.76
Third	no	41.75
	yes	13.36

Name: proportion, dtype: float64

BARRAS AGRUPADAS O PARALELAS

La función **crosstab()** permite resumir la relación entre dos variables categóricas en forma de **tabla de contingencia**. Además, permite calcular proporciones de manera directa:

```
pd.crosstab(df_titanic['class'], df_titanic['alive'], normalize = 'all').n
```

	alive	no	yes
class			
First	8.98	15.26	
Second	10.89	9.76	
Third	41.75	13.36	

El parámetro **normalize** permite definir cómo se calculan los porcentajes (**all**, **index**, **columns**).

GRÁFICO DE BARRAS APILADAS

Otra forma de mostrar la información anterior podría ser a través de un **gráfico de barras apiladas o *stackeadas***.

Para construir este tipo de visualización, tenemos que partir de la tabla de doble entrada que generamos en la *slide* anterior...

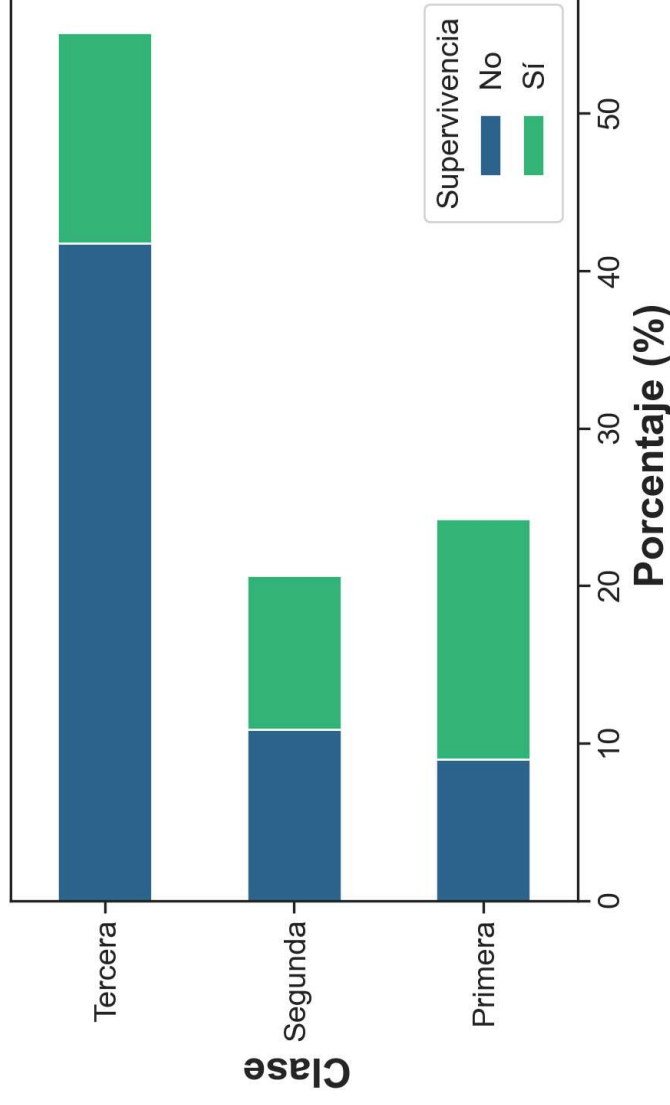
```
tabla1 = pd.crosstab(df_titanic['class'], df_titanic['alive'], normalize =
```

GRÁFICO DE BARRAS APILADAS

...y utilizar el método **plot.barh()** de **Pandas**, seteando el argumento **stacked = True** para mostrar barras apiladas:

```
tabla1.plot.barh(stacked = True) ;
```

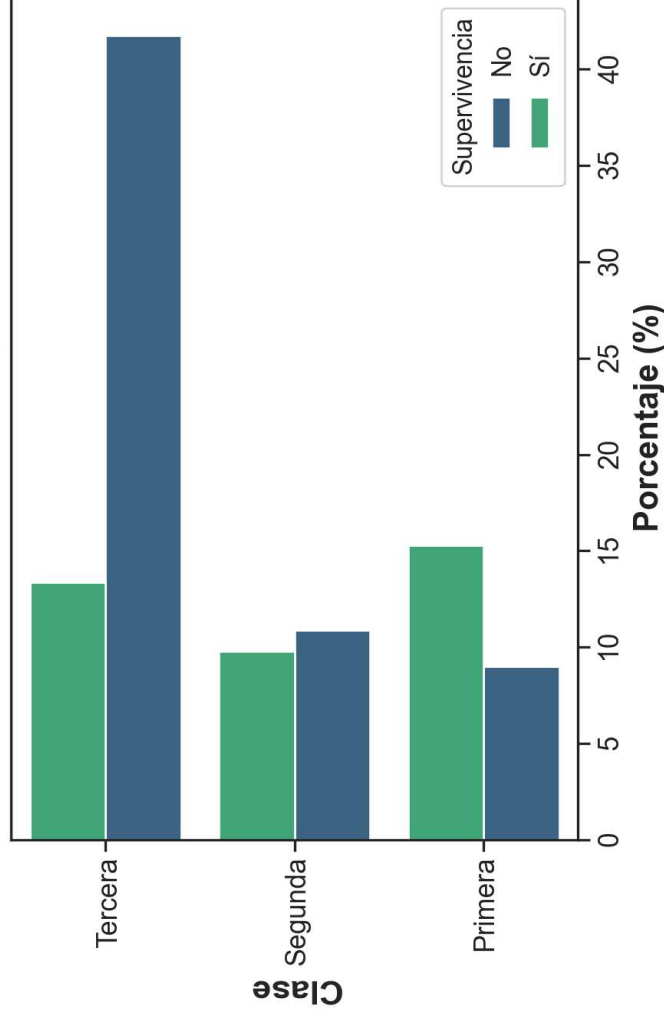
<Figure size 768x480 with 0 Axes>



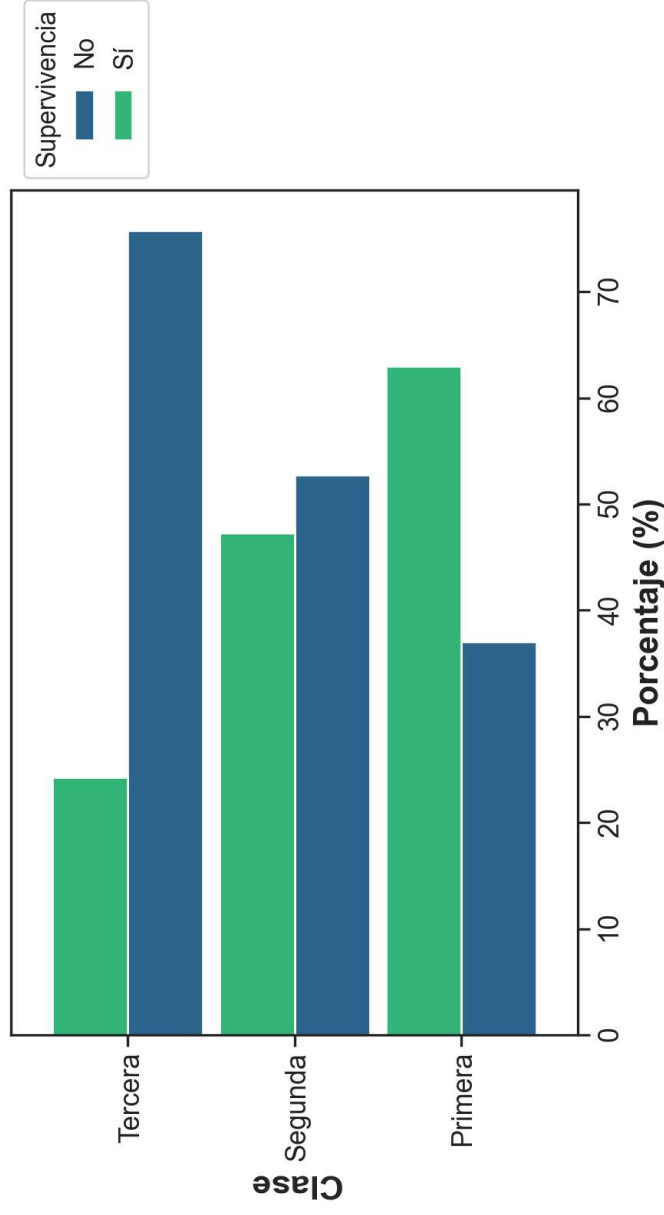
PARA PENSAR...

🤔 ¿Cuál es la diferencia entre estos dos gráficos, contruidos a partir del mismo dataset Titanic?

GRÁF. 1 - Barras paralelas



GRÁF. 2 - Barras paralelas

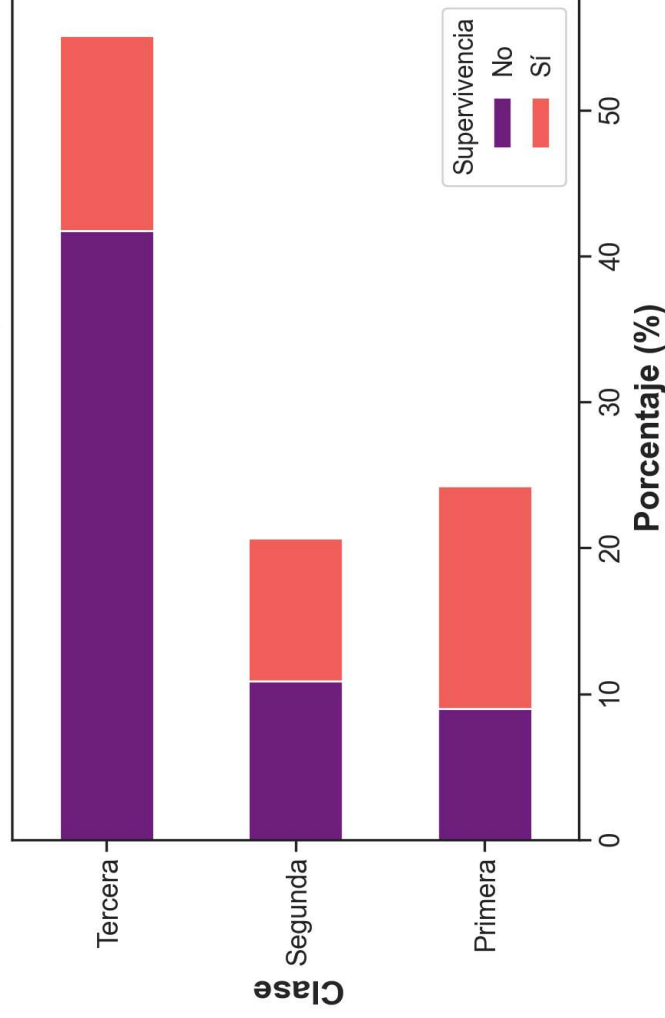


PARA PENSAR...

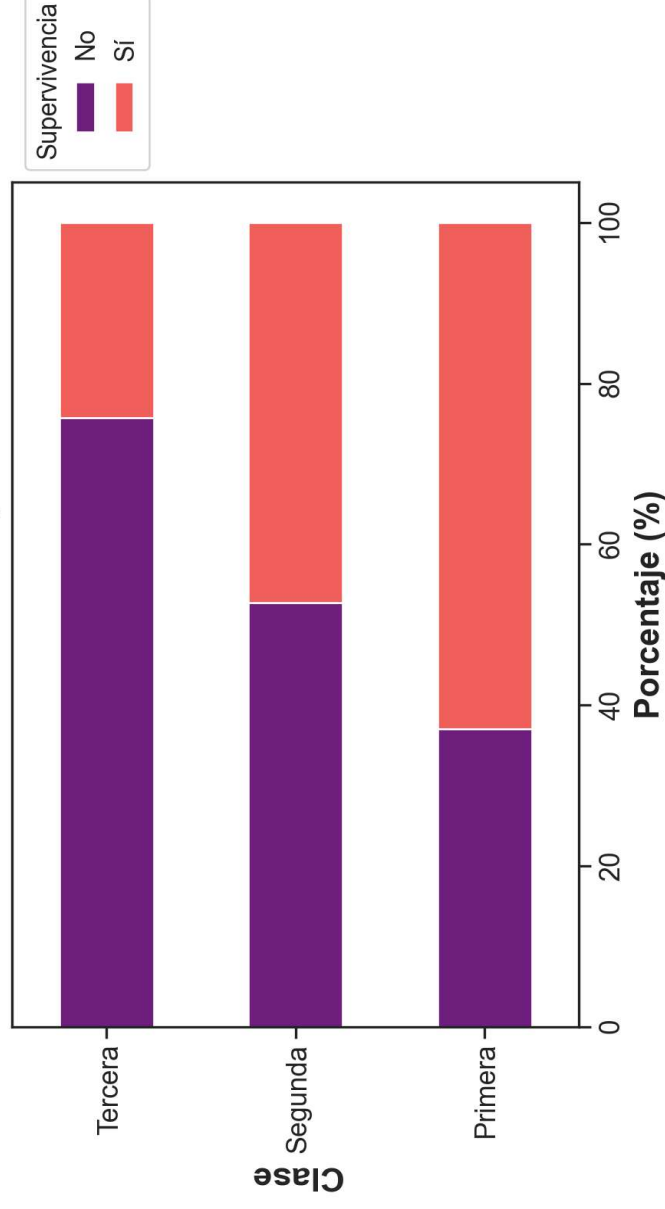


¿Se ve más claramente así? (el cambio en la paleta es sólo para diferenciar el tipo de gráfico)

GRÁF. 1 - Barras apiladas

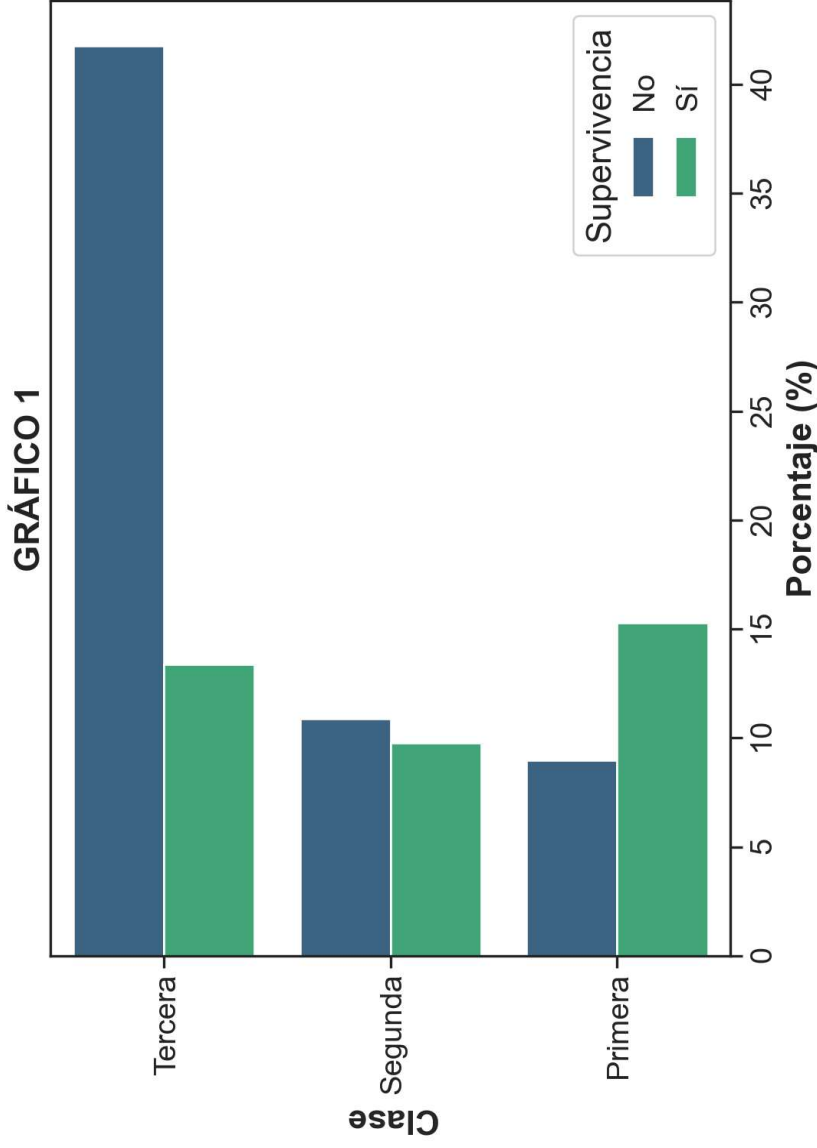


GRÁF. 2 - Barras apiladas



DISTRIBUCIÓN CONJUNTA/CONDICIONAL

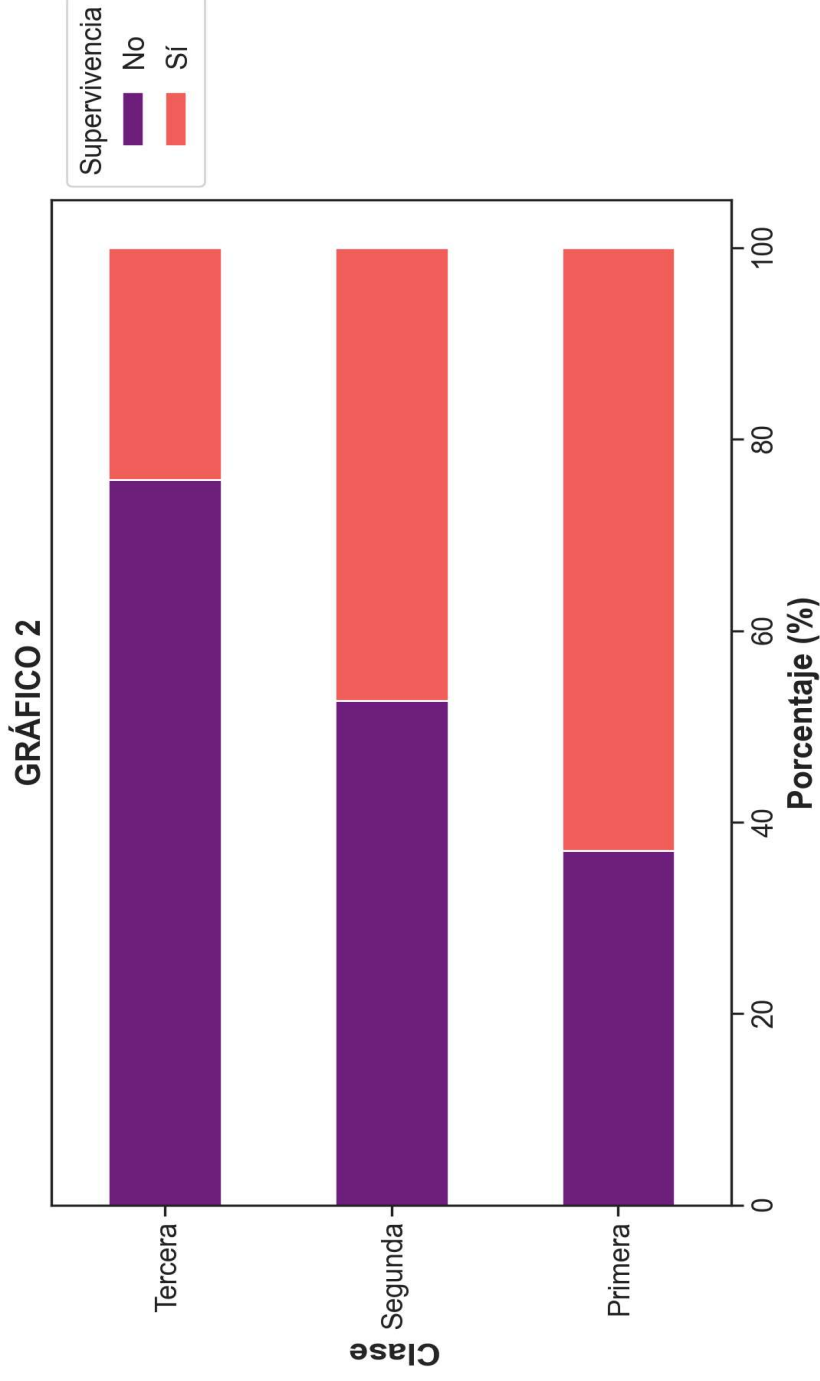
El primer gráfico muestra la **distribución conjunta** de las variables **class** y **alive** en el total de los datos, es decir, en el total de pasajeros que viajaban en el Titanic. Para llegar al 100%, hay que sumar los porcentajes de todas las barras.



El título podría ser: “Distribución de pasajeros del Titanic según **clase** y **supervivencia** al naufragio” .

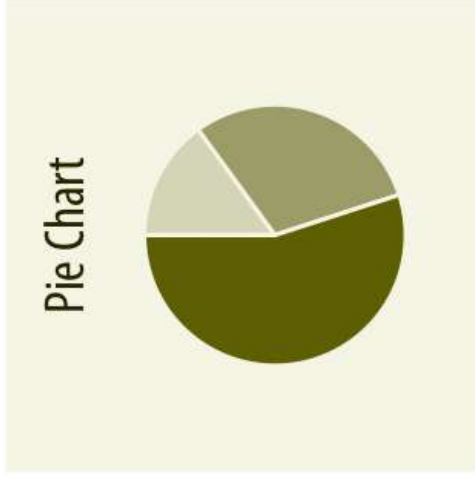
DISTRIBUCIÓN CONJUNTA/CONDICIONAL

El segundo gráfico muestra la **distribución condicional** de la supervivencia al naufragio dada la clase en la que viajaba el pasajero. Los porcentajes suman 100% dentro de cada una de las clases.



El título podría ser: **“Supervivencia al naufragio de los pasajeros según la clase en la que viajaban”**.

GRÁFICO DE SECTORES



La construcción del **gráfico de sectores** consiste en diagramar un círculo que representa al 100% de las unidades. El mismo se divide en **tantos sectores como categorías existan** y el **área** de cada uno de los sectores es proporcional al **porcentaje de unidades** que pertenecen a la categoría que representa.

GRÁFICO DE SECTORES

Podemos construir un gráfico de sectores para las observaciones de **class** generando previamente una tabla que contenga los porcentajes correspondientes a cada una de las tres categorías:

```
tabla_clases = df_titanic['class'].value_counts(normalize = True).mul(100)

print(tabla_clases)
```

	Clase	Porcentaje
0	Third	55.1
1	First	24.2
2	Second	20.7

GRÁFICO DE SECTORES

Luego, le damos forma al gráfico utilizando la función **pie()** de **matplotlib**:

```
plt.pie(data = tabla_clases, labels = 'Clase', x = 'Porcentaje', autopct =  
plt.legend(title = 'Clase', labels = ['Tercera', 'Primera', 'Segunda'], loc
```

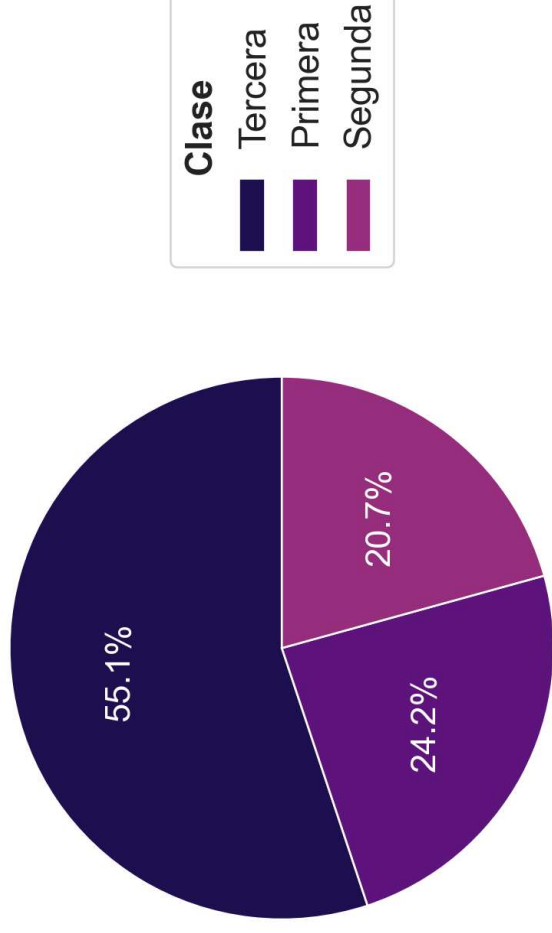


GRÁFICO DE SECTORES vs. BARRAS

Cuando tenemos un gran número de categorías y varias de ellas presentan proporciones similares, el gráfico de sectores puede volverse muy engorroso.

